KU LEUVEN

FACULTY OF ENGINEERING TECHNOLOGY

GROUP T LEUVEN CAMPUS

Design and Implementation of an Economical SatNOGS-Compatible Ground Station for the AetherSpace CubeSat

Stan COENE
Robbe LEHAEN

Supervisor: Prof. dr. ing. J. Vanhamel

Co-supervisor: Prof. dr. ir. V. De Smedt

Master's Thesis submitted to obtain the degree of Master of Science in Electronics-ICT programme Engineering Technology

Academic year 2025 - 2026

Samenvatting

Deze masterproef beschrijft het ontwerp, de implementatie en de veldvalidatie van een draagbaar, SatNOGS-compatibel grondstation voor de AetherSpace CubeSat op 433 MHz. Het station bestaat uit een Raspberry Pi 3A+ met twee subsystemen.

De rotator gebruikt een 3D-geprinte ASA-structuur met geïntegreerde geprinte lagerbanen en stalen kogels, gereduceerde DC-motoren (516:1 intern, tot 1376:1 totaal) en een op maat ontworpen RP2040-motorcontroller-PCB. Een PID-regellus haalt sub-0,5° elevatie- en 0–3° azimutfout, gevalideerd tijdens live passages. Elevatiekalibratie gebeurt via een ADXL345-accelerometer; azimut wordt gekalibreerd via het telefoonkompas of door op te starten vooraf op het noorden gericht.

Het RF-front-end is een 6-laags Pi HAT met twee TI CC1200 transceivers; het UHF-pad (432 MHz) is bestukt met een matching network afgeleid van de TI CC1200EM-referentie, terwijl het VHF-pad (144/145 MHz) gerouteerd is maar niet bestukt.

Een Python-stack op de Pi verzorgt passagevoorspelling, Dopplercorrectie, per-satelliet CC1200-herconfiguratie en indiening van SiDS-telemetrie. Een HTTPS-dashboard biedt laptop-gestuurd veldbeheer, met optionele telefoontoegang. Het station spreekt standaard EasyComm III via hamlib.

Over drie veldsessies werden ongeveer 210 satellietpassages gevolgd met een Siretta Oscar 44 Yagi. De CC1200-ontvangstketen is gevalideerd voor lokale testframes in korteafstandstests tussen twee CC1200-transceivers op de grond, maar geen protocolgeldige frames werden uit baan gedecodeerd. De link-budgetanalyse wijst een mastgemonteerde LNA aan als de belangrijkste resterende gevoeligheidskloof. De v2 MOTOR_RF_HAT (gecombineerde controller- en RF-print) is ontwerp-volledig maar bij indiening nog niet gefabriceerd.

Abstract

This thesis presents the design, implementation, and field validation of a portable, SatNOGS-compatible ground station targeting the AetherSpace CubeSat at 433 MHz. The station consists of a Raspberry Pi 3A+ with two subsystems.

The rotator uses a 3D-printed ASA structure with integrated printed bearing races and steel balls, geared DC motors (516:1 internal, up to 1376:1 total), and a custom RP2040 motor controller PCB. A PID loop achieves sub-0.5° elevation and 0–3° azimuth error, validated during live passes. Elevation homing uses an ADXL345 accelerometer; azimuth is calibrated via the phone compass or by powering up pre-aligned to north.

The RF front-end is a 6-layer Pi HAT carrying two TI CC1200 transceivers; the UHF (432 MHz) path is assembled with a matching network derived from the TI CC1200EM reference, while the VHF (144/145 MHz) path is routed but not populated.

A Python stack on the Pi handles pass prediction, Doppler correction, per-satellite CC1200 reconfiguration, and SiDS telemetry submission. An HTTPS dashboard provides laptop-based field control, with optional phone access. The station speaks standard EasyComm III over hamlib.

Across three field sessions, approximately 210 satellite passes were tracked with a Siretta Oscar 44 Yagi. The CC1200 receive chain was validated for local test frames in short-range ground-to-ground tests but no protocol-valid frames were decoded from orbit. Board-level interference (buck-converter harmonics, RP2040 digital noise, Pi GPIO emissions) lifts the effective noise floor approximately 30 dB above the thermal + NF baseline, dropping CC1200 sensitivity from its datasheet -120 dBm to approximately -105 dBm and leaving the AetherSpace link at -20.7 dB measured margin at 30° elevation. The link-budget analysis identifies a mast-mounted LNA as the dominant remaining sensitivity gap. The v2 MOTOR_RF_HAT (combined controller/RF board) is design-complete but not fabricated at submission.

Contents

1 Introduction **4**

1.1 Problem Statement 5

1.2 Research Questions 6

1.3 Scope and Division of Work 7

1.4 Thesis Outline 7

2 Background and Literature Review **8**

2.1 LEO Orbital Mechanics 8

2.2 The AetherSpace CubeSat 9

2.3 Link Budget 9

2.4 Doppler Shift 11

2.5 The SatNOGS Network 11

2.5.1 The EasyComm Protocol 12

2.5.2 Why a Distributed Network 12

2.5.3 CC1200 Deviation 12

2.6 Existing Designs 13

2.7 Comparison with Hanssens (2023) 13

3 System Design **14**

3.1 System Architecture 14

3.2 Mechanical Design 16

3.2.1 Structure 17

3.2.2 Bearings 17

3.2.3 Gearing 19

3.3 Motors 21

3.3.1 Torque budget 21

3.3.2 DC vs stepper 22

3.4 Motor Controller PCB . . . 23
3.5 Bring-Up and Hardware Debugging . . . 24
3.6 Cost . . . 25
3.6.1 v1 (deployed, field-validated) . . . 25
3.6.2 v2 (MOTOR_RF_HAT, design-complete, ready to order) . . . 26

4 RF Front-End **30**
4.1 RF Design Progression . . . 30
4.2 RF Devboard . . . 31
4.2.1 Architecture . . . 31
4.3 RF HAT Electrical Design . . . 32
4.3.1 System Overview . . . 32
4.3.2 Microcontroller Subsystem . . . 34
4.3.3 RF Transceiver Subsystem . . . 36
4.3.4 Communication Interfaces . . . 36
4.3.5 Power Distribution Network . . . 37
4.3.6 Summary . . . 38
4.4 RF HAT PCB Layout . . . 38
4.4.1 6-Layer Stackup Strategy . . . 38
4.4.2 Transmission Line Implementation . . . 39
4.4.3 Frequency-Dependent Layout Considerations . . . 41
4.4.4 Summary . . . 41
4.5 RF Matching Network . . . 41
4.5.1 CC1200 RF Interface Characteristics . . . 41
4.5.2 Matching Network Topology . . . 44
4.5.3 Signal Transformation Through the Matching Network . . . 45
4.5.4 Frequency-Selective Behavior . . . 46
4.5.5 Use of the Reference Design . . . 46
4.5.6 Summary . . . 46
4.6 V2 RF Sub-Circuit . . . 46

5 Firmware and Software **48**
5.1 Rotator Firmware: PID Control . . . 48
5.2 Quadrature Encoding . . . 49
5.3 Homing and Calibration . . . 49

5.4 Safety . . . 49
5.5 EasyComm Implementation . . . 50
5.6 RF Devboard Firmware . . . 50
5.7 RF HAT Firmware . . . 50
5.7.1 CC1200 Serial Mode and AX.25 G3RUH Decoding . . . 51
5.8 Software Architecture and Data Flow . . . 52
5.9 TLE Sources, Pass Prediction, and Profile Selection . . . 54
5.10 Doppler Correction Loop . . . 56
5.11 Status JSON Schema . . . 56
5.12 Operating Modes: Off / Track / Scan . . . 57
5.13 The HTTPS Dashboard . . . 57
5.13.1 Pre-Tracking View: Pass Selection . . . 58
5.13.2 Active-Tracking View: Live Telemetry . . . 59
5.13.3 Setup Wizard . . . 62
5.14 Background Services and Auto-Start . . . 62
5.15 SatNOGS Network Integration . . . 63
5.16 Field Deployment Workflow . . . 63

6 Results and Validation 65
6.1 Slew Speed . . . 65
6.2 PID Step Response . . . 65
6.3 Satellite Tracking Accuracy . . . 66
6.4 RF Front-End Measurements . . . 67
6.5 Packet Reception: Result and Interpretation . . . 69
6.5.1 Reception Modes Deployed . . . 70
6.5.2 Why no decoded frames . . . 70
6.5.3 What was observed: suggestive but not diagnostic . . . 70
6.5.4 Academic Conclusion . . . 73
6.5.5 AetherSpace-Specific Outlook . . . 74
6.6 Field Tests and Link Budget . . . 74
6.6.1 Firmware bugs surfaced by live passes . . . 75
6.7 Cost Comparison . . . 76

7 Conclusion and Future Work 77
7.1 Research Questions Answered . . . 77

7.2 Limitations 78
7.3 Future Work 79
7.4 Summary 80

A GPIO Pin Mapping **83**

B Communication Protocols **86**

C PID Tuning and Speed Test Data **90**

D Detailed Bill of Materials **92**

E GenAI code of conduct for students **96**

List of Figures

1.1 Assembled, field-deployed ground station tracking a pass. 4

3.1 Station architecture: Pi 3A+ host, MotorPCB, dual-CC1200 RF HAT, and power tree. 15
3.2 Assembled 3D-printed rotator, front view. 16
3.3 3D-printed deep-groove ball bearing close-up with 3 mm steel balls and press-in retainer. 18
3.4 Both printed bearings assembled, with the separate AZ inner race printed flat-down. 18
3.5 Printed herringbone gear set: AZ pair (15:40, 2.667:1) and EL pair (40:55, 1.375:1). 20
3.6 Parametric CAD source of the gear profile, generated in Onshape. 20
3.7 AZ torque measurement: 0.50 m lever, scale reads 0.71 kgf ⇒ 3.48 Nm. . . 22
3.8 EL torque measurement: 0.40 m lever, scale reads 1.10 kgf ⇒ 4.32 Nm. . . 22
3.9 KiCad 3D render of the v1 MotorPCB. 24
3.10 v1 RF_HAT : 6-layer Pi HAT containing the dual-CC1200 tranceivers. 25
3.11 v1 MotorPCB assembled prototype, mounted in the rotator electronics bay. 26
3.12 v2 MOTOR_RF_HAT (front): 6-layer Pi HAT merging motor control and dual-CC1200 RF. 27
3.13 v2 MOTOR_RF_HAT (back). 28

4.1 Two RF Devboards connected to usb 32
4.2 KiCad 3D render of the v1 RF Hat front 33
4.3 KiCad 3D render of the v1 RF Hat back 34

5.1 ADXL345 accelerometer breakout mounted on the elevation frame for EL homing. 49
5.2 End-to-end data flow: external sources, Pi services, browser. 53

5.3 Pre-tracking view: filter + mode segmented control (1), upcoming-passes list (2), recent-passes log (3). 58
5.4 Live-tracking dashboard: signal-strength sparkline (1), range/Doppler tile (2), pointing tile (3), RF status panel (4), sky plot (5), top status bar (6). . . 60
5.5 Close-up of the deployed station. 64

6.1 PID step response for a 90° commanded step. 66
6.2 Sky coverage heatmap: RSSI over AZ/EL across all tracked passes. 72
6.3 Doppler-correction S-curve applied during one pass. 73
6.4 Wide shot of the deployed station in the field. 74

7.1 24 V 6S LiPo pack intended for portable operation. 78

List of Tables

2.1 Link budget for the AetherSpace UHF downlink at 433 MHz across three canonical pass geometries. ‡Assumes a circularly polarised satellite antenna (turnstile); a linearly polarised dipole would give an orientation-dependent mismatch ranging from 0 dB (co-polarised) to $-\infty$ dB (cross-polarised). . . 10

2.2 Rotator comparison. All open-source entries except the two commercial units. ⋆AZ error excludes near-zenith passes where the required AZ rate exceeds the motor slew (§6.3). v2 PCB-only BOM is ∼€40. 13

3.1 Rotator gear ratios, encoder counts, and per-axis positional resolution. . . . 19

3.2 Rotator PCB pin assignments (as-deployed v1). 23

3.3 v1 (as-deployed) cost breakdown. 26

3.4 v2 (MOTOR_RF_HAT, projected) cost breakdown. Excludes antenna, coax, power supply, and tripod. 29

4.1 RF hardware iteration summary. 30

4.2 RP2040 Pin Assignment for v1 RF HAT 35

4.3 6-layer PCB stackup (JLC06121H-3313) with electrical function and material properties [31]. 39

6.1 Slew-speed measurements, unloaded, 24 V supply. 65

6.2 Tracking error binned by satellite peak elevation. 66

6.3 Devboard bench test results per frequency, averaged over both directions (50 packets each). LQI lower = better. 68

6.4 Cost comparison. v2 is design-complete but not yet fabricated; ∼€75/€100 are BOM projections (LCSC + JLCPCB, April 2026). 76

A.1 Motor controller Pico pin assignments (v1, as-deployed). 84

A.2 RF HAT Pico pin assignments. 85

B.1 EasyComm III commands (SatNOGS variant, hamlib model 204). 87

B.2 COBS binary protocol message types (RF HAT ↔ Pi). 89

C.1 PID and control-loop constants from `Firmware/rp2040-satnogs-rotator/src/config.h`. 90
C.2 Slew-speed measurements (single axis, unloaded, 24 V supply), as reported in §6.1. 91

D.1 MOTOR_RF_HAT on-board component cost breakdown. 93
D.2 Hand-soldered / off-batch items. 93
D.3 PCB fabrication cost (per board). 94
D.4 Off-board mechanical and computing components. 94
D.5 v2 grand-total cost (station core, excluding antenna, coax, power supply, and tripod). 94

Acronyms

AOS Acquisition of Signal. 1

ASA Acrylonitrile Styrene Acrylate. 1

AZ Azimuth. 1

BOM Bill of Materials. 1

CDC Communication Device Class (USB). 1

COBS Consistent Overhead Byte Stuffing. 1

COTS Commercial Off-The-Shelf. 1

CPR Counts Per Revolution. 1

CRC Cyclic Redundancy Check. 1

EL Elevation. 1

FDM Fused Deposition Modeling. 1

GFSK Gaussian Frequency Shift Keying. 1

GPIO General-Purpose Input/Output. 1

HAT Hardware Attached on Top (Raspberry Pi). 1

IMU Inertial Measurement Unit. 1

ISD Integrated Shutdown. 1

ISR Interrupt Service Routine. 1

LEO Low Earth Orbit. 1

LNA Low-Noise Amplifier. 1

LOS Loss of Signal. 1

MCU Microcontroller Unit. 1

PCB Printed Circuit Board. 1

PID Proportional–Integral–Derivative. 1

PLA Polylactic Acid. 1

PPR Pulses Per Revolution. 1

PWM Pulse-Width Modulation. 1

SDR Software-Defined Radio. 1

SiDS Simple Downlink Sharing. 1

SPI Serial Peripheral Interface. 1

TLE Two-Line Element set. 1

TSD Thermal Shutdown. 1

UART Universal Asynchronous Receiver/Transmitter. 1

UHF Ultra High Frequency. 1

UVLO Under-Voltage Lock-Out. 1

VHF Very High Frequency. 1

Chapter 1

Introduction

Figure 1.1: Assembled, field-deployed ground station tracking a pass.

1.1 Problem Statement

Geometry. A LEO satellite at 400 km altitude is visible from a given ground station for 5–10 minutes per 92-minute orbit. The total angular rate at zenith is bounded by

$$\dot{\theta}_{\max} = \frac{v}{h} = \frac{7.67\ \text{km/s}}{400\ \text{km}} = 0.01918\ \text{rad/s} = 1.10\ \text{deg/s}, \tag{1.1}$$

with $v = \sqrt{\mu/(R_E + h)} = 7.67$ km/s for $\mu = 3.986 \times 10^{14}$ m^3/s^2 and $R_E + h = 6771$ km [1]. The instantaneous azimuth rate exceeds 10°/s near zenith because a small displacement of the sub-satellite point maps to a large arc in azimuth.

Link. The Siretta Oscar 44 Yagi used in this work (9 dBi peak, 54°×48° 3-dB beamwidths at 434 MHz [2]) is required to close the link against 145.8 dB of free-space path loss at 433 MHz and 30° elevation (Chapter 2, Table 2.1). Its narrow beam sets the pointing-accuracy requirement on the rotator.

Cost. Commercial rotators cost €760–900 (Yaesu G-5500 [3]) to €1200–1300 (SPID RAS [4]); the SatNOGS v3 open-source rotator costs €300–500 in parts [5, 6]. The v2 MOTOR_RF_HAT BOM derived here (§3.6, design-complete) is approximately €40 for the combined controller/RF PCB, €75 for the full rotator hardware (PCB + motors + mechanics, excluding the Pi), and €100 for the station core (excluding antenna, coax, power supply, tripod).

Target. The AetherSpace CubeSat is a 3U spacecraft under development at KU Leuven for a heat-shield sample-return demonstration from LEO, launch ∼2030. Per AetherSpace specification (2026-03-05): 433 MHz UHF downlink, +14 dBm (≈25 mW), 2 dBi turnstile/dipole antenna, CC1200 native packet format. The link budget in Chapter 2 closes with positive theoretical margin only near zenith; at 30° elevation the theoretical margin is −5.7 dB (Table 2.1).

Radio choice. The station uses the CC1200 transceiver [7] at both ends. This deviates from the canonical SatNOGS receive chain (SDR + GNU Radio software demodulation) but eliminates per-satellite modulation-parameter uncertainty: both ends share the same SmartRF register set once the flight configuration is finalised.

A deliberate alternative, not a replacement

The canonical SatNOGS receive chain is a Raspberry Pi running the official `satnogs-client` daemon, an RTL-SDR v3 dongle feeding GNU Radio flowgraphs (`gr-satnogs`) through SoapySDR, and a hamlib-compatible rotator [8, 9]. That stack decodes a broad range

of amateur-satellite modulations (FSK, GFSK, AFSK, BPSK, LoRa chirp, CW, SSTV) in software, is maintained by the Libre Space Foundation, and remains the more flexible choice for a general-purpose amateur-radio ground station. The rotator produced by this thesis plugs into that stack unchanged over EasyComm III (hamlib model 204 [10]).

This thesis adopts the CC1200 hardware-packet-modem path instead, for three reasons.

1. AetherSpace-specific. AetherSpace carries a CC1200 on-board. Using the same transceiver at the ground end eliminates modulation-parameter matching uncertainty: a single SmartRF Studio register set loaded at both ends configures identical symbol rate, deviation, filter bandwidth, sync word, and packet framing. An SDR receiver would need those values re-measured or re-derived per satellite, per flight revision.

2. Educational scope. The thesis is a joint MSc in electronics. The CC1200 path exercises a full embedded-systems stack end-to-end: PCB design with controlled-impedance RF routing and matching networks (Chapter 4), RP2040 firmware with SPI drivers and a binary COBS protocol (§5.7, with protocol structure in Appendix B), Pi-side Python controlling CC1200 registers over UART (§5.8), and a closed-loop tracking rotator (Chapters 3 and 5).

3. Form-factor fit. A Pi 3A+ ships with 512 MB of LPDDR2 SDRAM [11]; with the default 64 MB GPU split, approximately 416 MB is available to userspace. The SatNOGS Docker stack targets $\geq$1 GB of RAM [8]. The Python stack described in §5.8 ($\sim$7900 LOC across the Pi packages listed in Appendix D) runs on the 3A+ within a 200–300 MB resident-set footprint measured with `top`, leaving margin for the HTTPS dashboard and systemd services.

Chapter 6 quantifies the sensitivity cost of this choice: with the Siretta Oscar 44 Yagi but without an external LNA, the CC1200 receive chain did not decode AetherSpace-class downlinks from orbit during the three field sessions. §7.3 identifies the hardware additions required to close the remaining sensitivity gap.

1.2 Research Questions

1. Can a satellite-tracking rotator be built for under €50 in materials?

2. What pointing accuracy is achievable with geared DC motors, 3D-printed structure, and encoder-based PID control?

3. Can the system integrate with the SatNOGS network using only the standard EasyComm/hamlib interface?

4. What are the empirical trade-offs of DC motors versus stepper motors for satellite tracking?

5. Is battery-powered portable operation feasible?

1.3 Scope and Division of Work

- Stan Coene: Mechanical rotator design, motor controller PCB (RP2040 + dual motor driver), rotator firmware, Raspberry Pi software stack, systemd services, web dashboard, field deployment workflow, system integration.
- Robbe Lehaen: RF front-end hardware (dual CC1200 transceiver HAT, UHF 433 MHz assembled + VHF 144/145 MHz routed), matching network design, RF HAT firmware (COBS protocol, SPI driver), CC1200 register configuration.

1.4 Thesis Outline

Chapter 2 derives the physics of LEO satellite communication from first principles: orbital mechanics, link budgets, Doppler shift, and the SatNOGS ecosystem.

Chapter 3 presents the system architecture, mechanical design, rotator controller PCB, power distribution, bring-up lessons, and cost analysis.

Chapter 4 describes the dual-CC1200 RF front-end HAT: PCB design, layout, and matching networks.

Chapter 5 covers the rotator firmware, RF HAT firmware, Raspberry Pi software stack, and field-deployment workflow.

Chapter 6 gives results and validation: tracking accuracy, RF measurements, field tests, and reception analysis. The link-budget derivation itself lives in Chapter 2 and is only referenced here.

Chapter 7 provides conclusions, limitations, and future work.

Chapter 2

Background and Literature Review

2.1 LEO Orbital Mechanics

For a circular orbit at altitude h above Earth's surface, the orbital velocity follows from equating gravitational attraction to centripetal acceleration [1]:

$$v = \sqrt{\frac{\mu}{R_E + h}} \tag{2.1}$$

with $\mu = GM_\oplus = 3.986 \times 10^{14}$ m^3/s^2 and $R_E = 6371$ km. For $h = 400$ km: $v = \sqrt{3.986 \times 10^{14} / 6.771 \times 10^{6}} = 7.669$ km/s. The orbital period is

$$T = 2\pi\sqrt{\frac{(R_E + h)^3}{\mu}} = 2\pi\sqrt{\frac{(6.771 \times 10^6)^3}{3.986 \times 10^{14}}} = 5545 \text{ s} = 92.4 \text{ min}. \tag{2.2}$$

The maximum slant range $d_{\max}$ at the geometric horizon (elevation $\epsilon = 0°$) for a spherical Earth is

$$d_{\max} = \sqrt{(R_E + h)^2 - R_E^2} = \sqrt{6771^2 - 6371^2} = 2293 \text{ km}. \tag{2.3}$$

Visible-arc geometry (the angle subtended at Earth's centre by the portion of the orbit above the station horizon) gives a theoretical maximum pass duration of

$$t_{\max} = T \cdot \frac{\alpha}{\pi} \approx 10.1 \text{ min at } h = 400 \text{ km, zenith pass,} \tag{2.4}$$

where $\alpha = \arccos(R_E/(R_E + h))$. Passes with practical elevations above 10° are shorter, typically 5–8 min in field data (Chapter 6).

The total angular rate as seen from the ground station reaches its bound at zenith:

$$\dot{\theta}_{\max} = \frac{v}{h} = \frac{7669}{400 \times 10^3} \text{ rad/s} = 1.10 \text{ deg/s}. \tag{2.5}$$

The *azimuth* rate, however, is unbounded as a pass approaches true zenith: for passes with closest-approach elevation above ∼80°, the AZ axis alone needs to move through a large arc in a short time, so instantaneous AZ rates can exceed 10°/s for a few seconds. This is a speed-limit concern for the mechanical rotator (§6.3) rather than an accuracy concern.

2.2 The AetherSpace CubeSat

AetherSpace is a 3U CubeSat developed at KU Leuven for a heat-shield sample-return demonstration from LEO, with a planned launch around 2030. Per the specification provided by the AetherSpace team (2026-03-05): UHF downlink at 433 MHz, TX power approximately +14 dBm (≈25 mW), TX antenna a turnstile or dipole with approximately 2 dBi gain, CC1200 native packet format, modulation and symbol rate not finalised at the time of writing. Hanssens (2023) on SatNOGS ground-station options [12] provides the closest prior KU Leuven work on the ground-station side.

2.3 Link Budget

The canonical link-budget scenario for the AetherSpace downlink is a 600 km-altitude pass at 30° peak elevation, giving a slant range of 1075 km at 433 MHz (formula $d = \sqrt{R_E^2 \sin^2\theta + h^2 + 2R_E h} - R_E \sin\theta$ with $R_E = 6371$ km). Two supporting rows bracket the geometry: a zenith pass at 400 km altitude (best case, 400 km slant) and a horizon pass at 10° elevation / 600 km altitude (worst case, 1932 km slant). All FSPL values use $L_{\text{fs}} = 20\log_{10}(d_{\text{km}}) + 20\log_{10}(f_{\text{MHz}}) + 32.45$.

Parameter	30°/600 km	Zenith/400 km	10°/600 km
TX power (AetherSpace)	+14.0 dBm	+14.0 dBm	+14.0 dBm
TX antenna (turnstile/dipole)	+2.0 dBi	+2.0 dBi	+2.0 dBi
FSPL at 433 MHz	−145.8 dB	−137.2 dB	−150.9 dB
Atmospheric loss (gaseous abs. + troposcintillation margin [13])	−0.5 dB	−0.3 dB	−1.0 dB
Polarisation mismatch (circ/lin)[‡]	−3.0 dB	−3.0 dB	−3.0 dB
RX antenna (Oscar 44, 9 dBi [2])	+9.0 dBi	+9.0 dBi	+9.0 dBi
Feedline loss (3 m RG58)	−0.9 dB	−0.9 dB	−0.9 dB
Pointing loss	−0.5 dB	−0.3 dB	−0.5 dB
Received power at CC1200	**−125.7 dBm**	**−116.7 dBm**	**−131.3 dBm**
CC1200 sensitivity [7] (2.4 kbps 2-GFSK; interp. from −123 dBm at 1.2 kbps at −3 dB/octave: noise bandwidth scales linearly with symbol rate under matched-filter conditions)	−120 dBm	−120 dBm	−120 dBm
Theoretical margin	**−5.7 dB**	**+3.3 dB**	**−11.3 dB**
CC1200 effective sensitivity (indoor best-case, no LNA; field floor ≈−96 dBm)	−105 dBm	−105 dBm	−105 dBm
Measured margin	**−20.7 dB**	**−11.7 dB**	**−26.3 dB**

Table 2.1 Link budget for the AetherSpace UHF downlink at 433 MHz across three canonical pass geometries. [‡]Assumes a circularly polarised satellite antenna (turnstile); a linearly polarised dipole would give an orientation-dependent mismatch ranging from 0 dB (co-polarised) to $-\infty$ dB (cross-polarised).

Two observations follow. First, even at the datasheet CC1200 sensitivity the AetherSpace link closes with positive margin only at high elevations; mid-elevation and horizon geometries are negative. Second, the assembled RF HAT's noise floor is elevated well above the thermal+NF limit by board-level interference (LMR51420 buck harmonics, RP2040 digital noise, Pi GPIO emissions), pushing the measured margin deeply negative across all geometries.

The two sensitivity figures in the table reflect two different conditions. The −105 dBm used in the measured-margin row is the best-case figure, observed in the quietest indoor lab environment where the CC1200 RSSI reads as low as −105 dBm (§6.4); even under these conditions the link margin is −20.7 dB. The −96 dBm canonical figure (`captures/evidence/signal_evidence.md`; §6.5) is the typical outdoor noise floor from the field sessions, sitting approximately 30 dB above the expected kTB+NF limit of −127 dBm; at this floor the received signal of −125.7 dBm is buried 29 dB in noise, making decode without an external LNA impossible regardless of link geometry. The full Friis cascade analysis of the LNA improvement is given in the link-budget reference document main-

tained alongside the thesis (`ThesisPaper/draft_link_budget.md`).

2.4 Doppler Shift

A satellite moving with radial velocity v_r relative to the ground station shifts the received carrier by $\Delta f = v_r \cdot f_0 / c$. The radial velocity reaches its maximum at the geometric horizon, where the full orbital velocity projects onto the line of sight:

$$v_{r,\max} \approx v = 7669 \text{ m/s}. \tag{2.6}$$

At 433 MHz the magnitude of the peak Doppler shift (satellite at horizon, full orbital velocity along the line of sight) is

$$|\Delta f|_{\max} = \frac{7669}{3 \times 10^8} \cdot 433 \times 10^6 = 11.07 \text{ kHz}, \tag{2.7}$$

giving a total peak-to-trough excursion of ± 11 kHz across a full pass (positive at approach, negative at recession, zero at TCA).

At 145 MHz (representative VHF band centre; no uplink frequency has been defined for AetherSpace at the time of writing):

$$|\Delta f|_{\max} = \frac{7669}{3 \times 10^8} \cdot 145 \times 10^6 = 3.71 \text{ kHz} \quad (\pm 3.7 \text{ kHz peak-to-trough}). \tag{2.8}$$

The Doppler rate $d(\Delta f)/dt$ peaks at closest approach. The station software applies Doppler correction by writing the CC1200 `FREQOFF` register at 2 Hz (§5.8). A worst-case rate of approximately 200 Hz/s at 433 MHz near TCA means the residual uncompensated drift between 2 Hz updates is of order 100 Hz, which is small relative to the 10 kHz RX filter bandwidth used for 2.4 kbps 2-GFSK reception.

2.5 The SatNOGS Network

SatNOGS (Satellite Networked Open Ground Station) originated at Hackerspace.gr in Athens during the 2014 NASA Space Apps Challenge and won that year's Hackaday Prize, which funded the Libre Space Foundation [8]. A detailed ecosystem history appears in [12], which this thesis continues; the summary below is limited to what is load-bearing for the subsequent chapters.

The ecosystem has four components [8, 14, 15]:

SatNOGS Network (`network.satnogs.org`): observation scheduling across stations based on TLE pass predictions.

SatNOGS DB (`db.satnogs.org`): telemetry database, accepts decoded frames over the SiDS (Simple Downlink Sharing) HTTP API.

SatNOGS Client : Python daemon running on each station, polls Network for scheduled observations, drives the rotator via hamlib/`rotctld`, drives the SDR receiver via SoapySDR + GNU Radio, uploads results.

SatNOGS Rotator : optional reference rotator (3D-printed, NEMA17 bipolar steppers, belt drive, Arduino controller) [16].

Rotator communication goes through hamlib [9]: the `rotctld` daemon exposes a TCP interface on port 4533 speaking the hamlib `rotctl` protocol, and internally translates those commands to EasyComm III ASCII on the serial link to the controller. Any controller that speaks EasyComm III integrates without modification, as hamlib model 204 [10].

2.5.1 The EasyComm Protocol

EasyComm is an ASCII protocol developed by Chris Jackson (G7UPN) for antenna tracking [10]. The SatNOGS variant uses hamlib model 204 (EasyComm III), with commands summarised in Appendix B.

2.5.2 Why a Distributed Network

A single station contacts a LEO satellite for at most $\sim$11% of each orbit ($t_{\max}/T \approx 10.1/92.4 = 10.9\%$ at 400 km). For mission-critical telemetry, one station is insufficient; distributing stations worldwide gives near-continuous coverage. Earlier attempts (the Japanese Ground Station Network at the University of Tokyo, and ESA's GENSO project, started 2007 and cancelled by 2014) coupled stations through a centralised SOAP-over-SSL software stack, which became the lock-in that prevented open scaling [12]. SatNOGS accepts any hardware that speaks EasyComm over serial and uses an SDR receiver.

2.5.3 CC1200 Deviation

The standard SatNOGS receive chain is SDR-based. An RTL-SDR v3 dongle feeds raw IQ samples through GNU Radio flowgraphs (`gr-satnogs`) via SoapySDR; demodulation, sync detection, and CRC checking happen in software. This stack is flexible at the cost of compute requirements (GNU Radio + Docker, targeting $\geq$1 GB of RAM [17]).

This thesis substitutes a CC1200 hardware packet modem for the SDR front-end. The CC1200 performs modulation/demodulation, sync-word detection, and CRC checking in hardware, producing decoded packets rather than raw IQ. As a consequence, the standard `satnogs-client` pipeline (built around GNU Radio) cannot be used as-is; the station submits decoded frames directly to the SatNOGS DB via the SiDS API. The trade-off

is reduced modulation flexibility (the CC1200 must be configured to match each satellite's transmit parameters) in exchange for alignment with AetherSpace (which carries a CC1200) and lower compute requirements on the Pi 3A+ [11].

2.6 Existing Designs

System	Type	Cost (€)	Accuracy	Ref.
Yaesu G-5500	Commercial	760–900	$\sim$1°	[3]
SPID RAS	Commercial	1200–1300	$\sim$1°	[4]
SatNOGS v3	Stepper, belt	300–500	$\sim$1°	[5, 6]
SuperAntennaz	Stepper	$\sim$600	$\sim$1°	[18]
This work (v2)	**DC + gears**	**$\sim$75**	**$<$0.5° EL / 0–3° AZ***	§3.6

Table 2.2 Rotator comparison. All open-source entries except the two commercial units. *AZ error excludes near-zenith passes where the required AZ rate exceeds the motor slew (§6.3). v2 PCB-only BOM is $\sim$€40.

Most university ground stations (UPSat/University of Patras [19], TU Delft/Delfi-C3 [20], AraMiS/Politecnico di Torino [21]) buy commercial rotators and focus on the RF or software layers. Custom mechanical rotator design for university CubeSat ground stations is comparatively rare; SuperAntennaz [18] is an open-source outlier targeting heavy-antenna loads ($>$10 kg) using steppers.

2.7 Comparison with Hanssens (2023)

This thesis continues the 2023 masterproef by Serge Hanssens [12], which surveyed the SatNOGS ecosystem, assessed hardware options (rotators, SDRs, antennas), and tested a Stepperdrive MSD-50-5.6D stepper motor driver for azimuth rotation.

Hanssens' load-bearing finding for the present thesis is that the stepper motor he tested did not deliver sufficient torque for reliable azimuth rotation under wind loading [12]. He measured the required torque for azimuth movement on a cantilevered Yagi and concluded that a higher-torque motor or higher gear ratio was needed. This directly informed the choice in the present work of geared DC motors (FAPG36-555-EN, 516:1 internal planetary gearbox) with a further 2.667:1 external reduction, covered in §3.3.

Hanssens' research questions were survey-oriented; the present work's are implementation-oriented, and the two studies therefore compare as scope rather than as competing results.

Chapter 3

System Design

3.1 System Architecture

Three subsystems connected through a Raspberry Pi 3A+:

1. Rotator: 2-axis AZ/EL, DC motors, custom RP2040 motor PCB. USB CDC serial, EasyComm III.
2. RF front-end: Pi HAT, 2$\times$ CC1200 populated only on UHF (433 MHz); VHF (144/145 MHz) path routed but not populated. Second RP2040, UART link, COBS binary protocol.
3. Station computer: Pi runs `rotctld` (hamlib), `station.py`, `dashboard.py`. Pass prediction, Doppler correction, telemetry submission.

Architecture keeps rotator and RF independent: either can be tested, replaced, or upgraded alone.

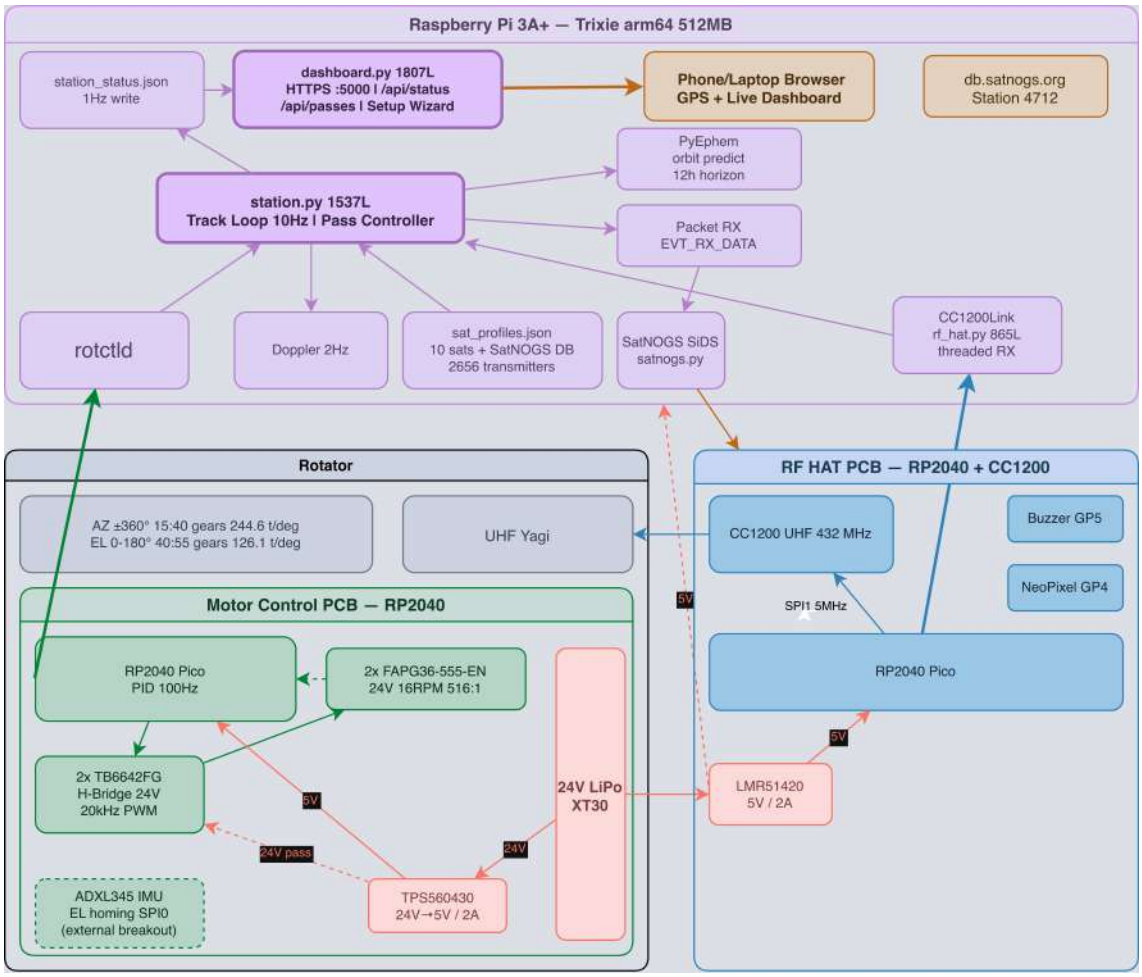

Figure 3.1: Station architecture: Pi 3A+ host, MotorPCB, dual-CC1200 RF HAT, and power tree.

Design constraints

- Portability. 1/4”-20 UNC tripod thread on the base. Field setup time measured across three sessions: approximately 4 minutes from unpacking to first tracked pass (§6.6).
- Zero-SSH workflow. All field operations (GPS entry, north alignment, pass selection, park, shutdown) run through the HTTPS dashboard. The UI targets a laptop browser; a phone browser also works for quick field tasks such as supplying a GPS fix.
- Cost target. v2 MOTOR_RF_HAT BOM (§3.6, Appendix D): ∼€40 combined controller/RF PCB, ∼€75 rotator hardware, ∼€100 station core (excluding antenna, coax, power supply, tripod).
- Licensing (intended). Hardware under CERN-OHL-S v2; firmware and Python stack

under MIT. Formal `LICENSE` files are added with the open-source release.

3.2 Mechanical Design

The rotator is designed in Onshape and printable on consumer FDM machines with a 220×220 mm bed (the largest single printed part is 205×160×120 mm). All structural prints are in ASA (Acrylonitrile Styrene Acrylate). ASA is chosen over PLA for two properties: (i) UV stability from the acrylate rubber phase (a polymer-science property of ASA copolymers; ASA substitutes the acrylate rubber for the butadiene phase of ABS, eliminating the UV-degradation vector), and (ii) a glass transition $T_g \approx 100$°C/ Vicat softening $\approx$ 105°C, compared to $T_g \approx 60$°C for PLA; this margin avoids thermal deformation in direct sunlight. PETG was considered and rejected because of lower interlayer adhesion and creep under sustained load compared with ASA at equivalent print settings.

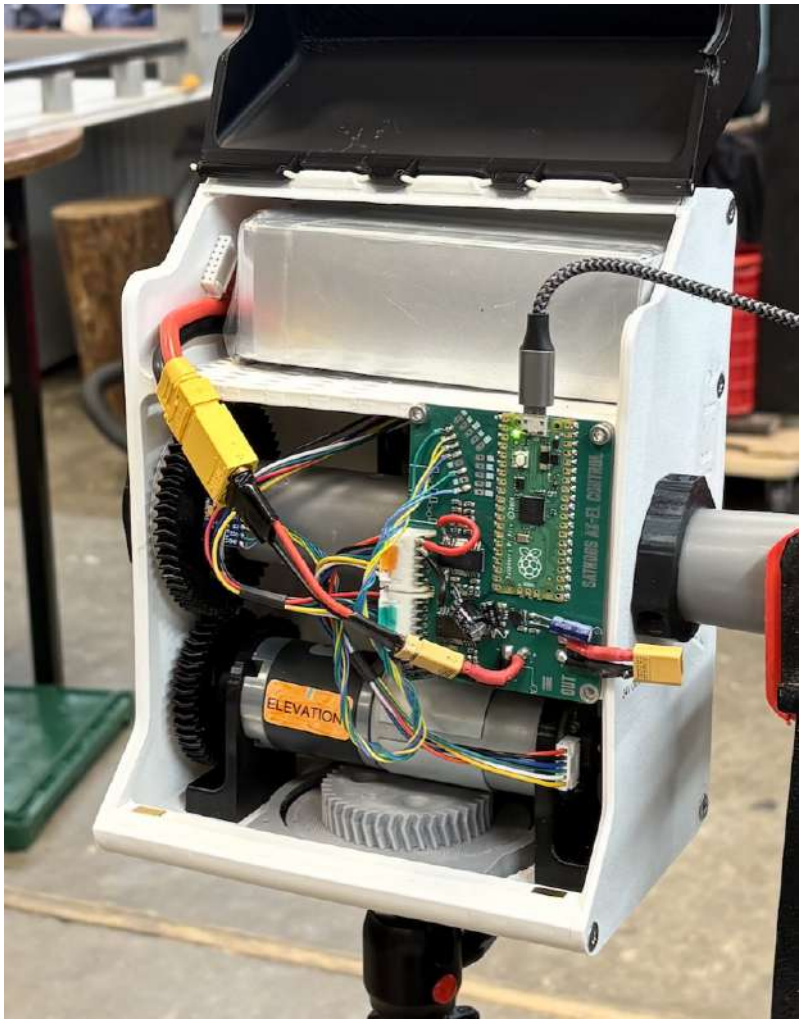

Figure 3.2: Assembled 3D-printed rotator, front view.

3.2.1 Structure

The chassis consists of a base plate with an integrated azimuth bearing race, two side plates forming the elevation frame, and a removable hood covering the electronics compartment. The hood attaches via a 3D-printed hinge, giving tool-free access to cable connections during bring-up.

The station mounts to any standard camera tripod through a 1/4”-20 UNC brass heat-set insert in the base plate. Using the tripod standard rather than a custom mount lets the station share hardware with existing camera tripods in the user’s toolkit.

The elevation axis is a PVC pipe passing through both side plates. The elevation gear is clamped directly onto this pipe by integrated 3D-printed screw clamps. The clamp geometry lets the EL gear be slid along the pipe during assembly to set mesh with the motor pinion, without set screws or adhesives.

All fasteners are M4$\times$10 mm countersunk screws for structural joints and M3$\times$8 mm for motor mounts. The assembly uses no adhesives (every joint is either a press-fit, a screw fastener, or a clamp), so the station disassembles for transport or repair without destructive separation.

3.2.2 Bearings

Both axes use 3D-printed deep-groove ball bearings with COTS 3 mm steel balls. The elevation outer race is printed in place into the side plate. The azimuth outer race is a separate screw-in part: the raceway has to be printed flat-down on the build plate to avoid layer stairstepping on the running surface, which prevents printing it as a feature of the base plate. Inner races are separate rings that double as the rotating element (the main disk for AZ); a press-in 3D-printed retainer holds the ball complement.

The trade-off is surface finish: printed races have higher R_a than machined steel bearings. Two properties of the rotator keep this acceptable: (i) the operating speed at the output is bounded by the motor output divided by the external ratio (approximately 6 RPM on the AZ axis via the 15:40 reduction, approximately 11 RPM on EL via the 40:55 reduction, at 95 % duty; §6.1), so contact-surface sliding velocity is low; and (ii) the bearing preload is low because the herringbone gear pair below imposes negligible axial thrust load (§3.2.3). Endurance characterisation beyond the field sessions is future work.

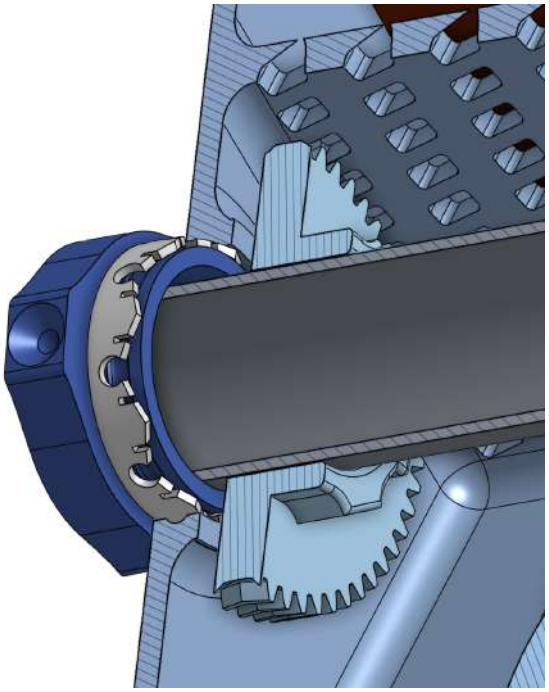

Figure 3.3: 3D-printed deep-groove ball bearing close-up with 3 mm steel balls and press-in retainer.

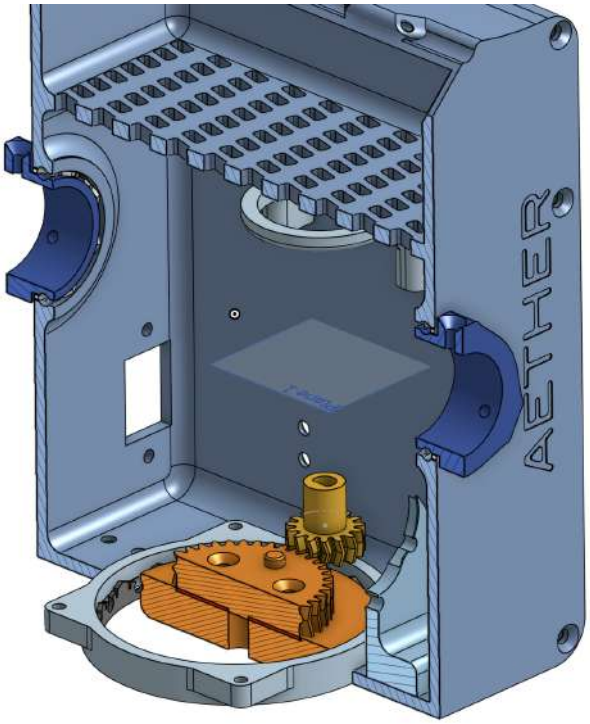

Figure 3.4: Both printed bearings assembled, with the separate AZ inner race printed flat-down.

3.2.3 Gearing

All gears use a double-helical (herringbone) tooth profile with a 15° helix angle, 4/3 mm normal module, and 8 mm face width. The herringbone profile cancels the axial thrust generated by each helical half on a single-helical gear, which matters here because the printed radial deep-groove bearings tolerate much less axial than radial load. The 4/3 mm module is non-standard in the ISO 53 preferred series [22] but gives integer reference diameters (e.g. 15 teeth $\times$ 4/3 mm = 20 mm) that print cleanly on FDM at 0.2 mm layer height.

The gear-ratio bookkeeping:

Axis	Teeth	Ext. ratio	Internal gearbox	Total ratio	CPR	Ticks/°	Travel
AZ	15:40	2.667:1	516:1	1376:1	64	244.6	$\pm$360°
EL	40:55	1.375:1	516:1	709.5:1	64	126.1	0–180°

Table 3.1 Rotator gear ratios, encoder counts, and per-axis positional resolution.

Ticks per degree at the output is derived from the encoder count per motor revolution multiplied by the total reduction, divided by 360:

- AZ: ticks/deg $= \dfrac{64 \times 516 \times (40/15)}{360} = \dfrac{64 \times 516 \times 2.667}{360} = \dfrac{88\,064}{360} = 244.6$ ticks/deg, giving a positioning resolution of $1/244.6 = 0.00409°$ per tick.
- EL: ticks/deg $= \dfrac{64 \times 516 \times (55/40)}{360} = \dfrac{64 \times 516 \times 1.375}{360} = \dfrac{45\,408}{360} = 126.1$ ticks/deg, giving 0.00793° per tick.

Gears press-fit directly onto the motor's 6 mm D-shafts with additional set-screws.

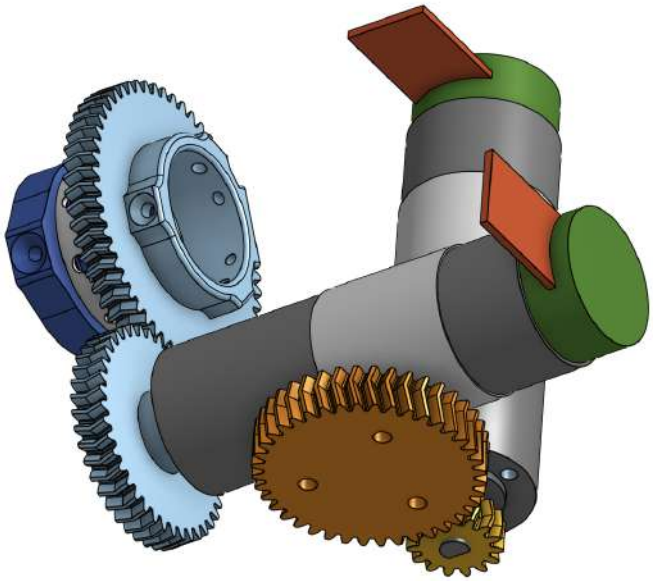

Figure 3.5: Printed herringbone gear set: AZ pair (15:40, 2.667:1) and EL pair (40:55, 1.375:1).

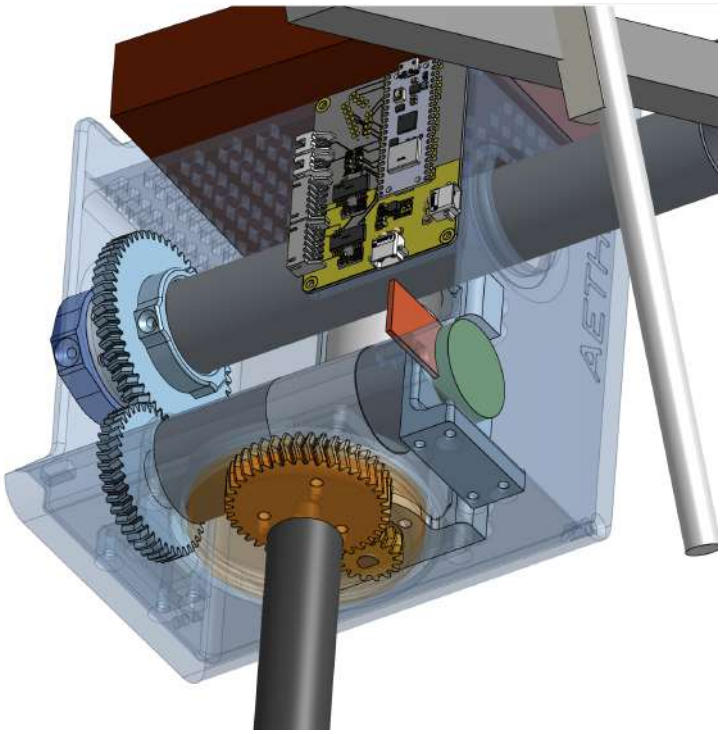

Figure 3.6: Parametric CAD source of the gear profile, generated in Onshape.

3.3 Motors

The axis motor is the FAPG36-555-EN (FONEACC): a 24 V brushed DC motor with a 516:1 internal planetary gearbox [23]. Manufacturer spec: approximately 16 RPM no-load at 24 V, nominal output torque 4 Nm, 6 mm D-shaft output. A Hall-effect quadrature encoder is mounted on the motor shaft (before the gearbox). The manufacturer datasheet specifies 12 PPR (48 CPR in 4$\times$ decode); the purchased units empirically measure as 16 PPR (64 CPR in 4$\times$ decode), verified by matching firmware encoder counts against known commanded axis rotations during bring-up. The 6 mm D-shaft press-fits directly into the printed gear.

3.3.1 Torque budget

Hanssens [12] identified insufficient stepper torque as the failure mode of his azimuth rotation tests. The DC-motor path chosen here multiplies the manufacturer output torque by the external gear ratio:

- AZ: 4 Nm $\times$ 2.667 $=$ 10.7 Nm at the output shaft (manufacturer spec $\times$ external ratio).
- EL: 4 Nm $\times$ 1.375 $=$ 5.5 Nm at the output shaft.

Measured output torque. The assembled rotator was bench-measured with a lever-arm-plus-kitchen-scale setup. A rigid arm attached to the axis output pushes down on the scale at a known radius while the axis is commanded to rotate at 95 % duty; the peak scale reading gives the output force.

- AZ: 0.71 kgf at 0.50 m lever $\Rightarrow$ $T_{\mathrm{AZ}} = 0.71 \cdot 9.81 \cdot 0.50 = 3.48$ Nm (Fig. 3.7).
- EL: 1.10 kgf at 0.40 m lever $\Rightarrow$ $T_{\mathrm{EL}} = 1.10 \cdot 9.81 \cdot 0.40 = 4.32$ Nm (Fig. 3.8).

Measured EL reaches approximately 80 % of the spec-derived 5.5 Nm. Measured AZ reaches approximately 33 % of the spec-derived 10.7 Nm; the AZ rig lets the rotor continue to move as the scale deflects, so the reading is the rotating-under-load torque rather than the stalled peak. Both measured values exceed what the Siretta Oscar 44 Yagi (mass $\sim$0.6 kg) requires at the lever of the assembled antenna.

Figure 3.7: AZ torque measurement: 0.50 m lever, scale reads 0.71 kgf ⇒ 3.48 Nm.

Figure 3.8: EL torque measurement: 0.40 m lever, scale reads 1.10 kgf ⇒ 4.32 Nm.

3.3.2 DC vs stepper

The choice of DC motors over stepper motors was informed by Hanssens' torque test [12] and by three properties of the DC-motor-plus-high-ratio-gearbox path:

1. Zero idle current with mechanical position holding. A brushed DC motor draws no current when stationary, and a 516:1 reduction makes back-driving from the output side require $516\times$ the static friction torque of the unloaded motor rotor, which for a planetary gearbox of this ratio practically prevents back-drive under wind gusts. The station is unpowered at the motor terminals for the $\sim$90 % of the orbit it is idle (about 80 minutes out of a 92-minute orbit, per §2.1). In contrast, a stepper requires continuous holding current (typically 0.8–1.5 A per phase at rated current per the Pololu stepper catalogue [24]) to resist back-drive; a de-energised stepper has no holding torque.

2. Torque retained at low speed. The 516:1 internal gearbox plus the external stage delivers the torque budget listed above across the full output-speed range used for tracking (0–35°/s). Steppers lose torque above their pull-out speed and require microstepping to produce smooth motion, which further reduces available torque per step.

3. Closed-loop position verification. The quadrature encoder on the motor shaft gives continuous position feedback. A stepper skipping a step (wind gust, obstruction, or torque-curve excursion) silently diverges commanded from actual position unless an external encoder is added; with the encoder already in place here, the PID loop sees any discrepancy within one control period.

The trade-off is firmware complexity: the DC-motor path requires a PID loop and quadrature-decode ISRs (§5.1, §5.2). The encoder feedback also makes safety features possible (runaway detection, soft-limit enforcement) that a stepper-without-encoder cannot implement.

No slip ring is fitted on the azimuth axis. The firmware tracks cumulative rotation and enforces soft limits at $\pm 360^{\circ}$, so the antenna cable never wraps more than one full turn. Between passes, the `PARK` command returns the antenna to true-north / zero position via the shortest path, which unwraps any accumulated wrap before the next observation (§5.5).

3.4 Motor Controller PCB

The v1 MotorPCB is a 4-layer KiCad 9 design with 13 hierarchical sub-sheets, JLCPCB assembled from LCSC basic parts.

Components:

- RP2040 (Pi Pico module): EasyComm firmware, PID, encoder ISRs.
- 2× TB6642FG [25]: 50 V absolute-max / 4.5 A peak H-bridge. IN1/IN2 + PWM. 20 kHz PWM frequency.
- ADXL345 [26]: 3-axis accelerometer on SPI0, gravity vector for EL homing.
- Encoder inputs: 4 GPIO with IRQ, 4× quadrature decoding. RC filter caps removed (see §3.5).
- NeoPixel: optional WS2812B status output: idle = green, tracking = blue, on-target = cyan, fault = red.

The fielded v1 firmware uses GP16 for AZ IN1 after the GP15 damage described in §3.5; the schematic's original GP15 assignment is retained only as bring-up history.

Function	AZ pins	EL pins
Motor IN1/IN2/PWM	GP16/GP14/GP22	GP28/GP27/GP21
Encoder A/B	GP11/GP10	GP12/GP13

Table 3.2 Rotator PCB pin assignments (as-deployed v1).

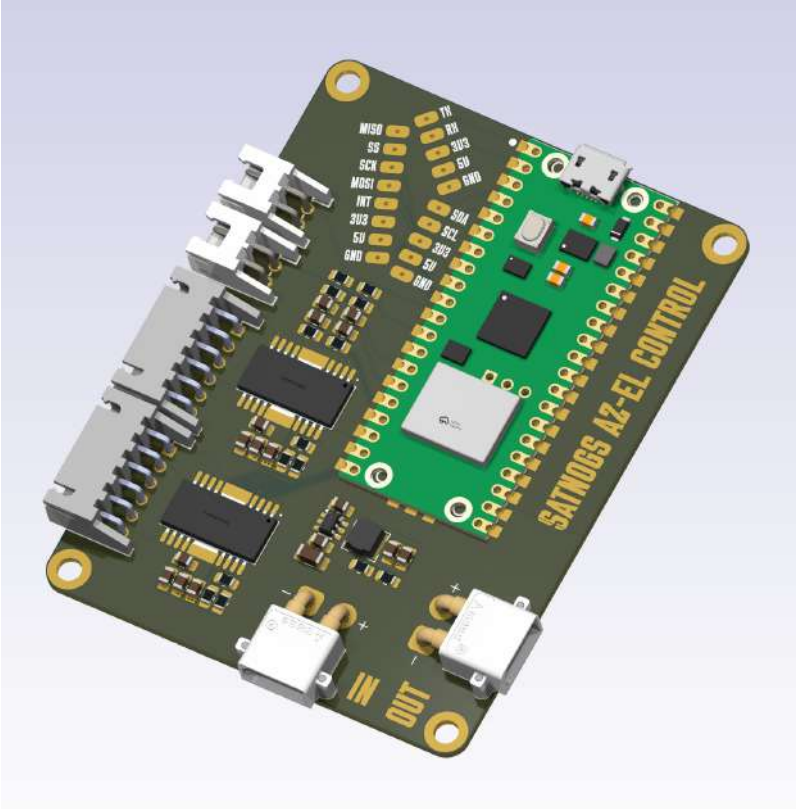

Figure 3.9: KiCad 3D render of the v1 MotorPCB.

3.5 Bring-Up and Hardware Debugging

The first-revision MotorPCB shipped with three defects, all fixed with bodge wires and reflected in the v2 design:

- Missing EL driver traces. OUT1/OUT2 on the elevation TB6642FG were unrouted; bodged through unused NC pins (6, 11).
- Encoder RC filters (C1–C4). 100 nF caps on the Hall-sensor lines low-passed the signal into unusability; removed. Debouncing is handled in firmware.
- AZ driver blown by bootloader cycling. The 1200-baud bootloader trick with 24 V connected damaged the TB6642FG IN1 ESD diode and the GP15 pad; driver replaced and IN1 rerouted to GP16.

The v2 MOTOR_RF_HAT corrects all three natively: routed EL outputs, no encoder caps, and 100 Ω series resistors (R11–R14) on every GPIO-to-driver line. Merging the motor controller and RF front-end onto one Pi HAT also removes the USB cable and separate 5 V path.

3.6 Cost

Two hardware configurations are costed: the as-deployed v1 setup used for field validation (separate MotorPCB + RF HAT, connected via USB and UART), and the v2 combined MOTOR_RF_HAT which is the thesis's final hardware deliverable (design-complete, routed, BOM finalised, ready to order but not yet fabricated at the time of writing). The strict "under €50 full rotator" target is not met once motors and mechanics are included; v2 reaches ∼€40 for the combined controller/RF PCB, ∼€75 for the rotator hardware excluding the Pi, and ∼€100 for the station core. v1 existed to validate the architecture and firmware and to allow both sides (RF and rotator) to be developed separately.

Component prices from the LCSC BOM tool (April 2026), bare PCB from JLCPCB (5-piece order). Full BOM in Appendix D and `ThesisPaper/bom_complete.csv`.

3.6.1 v1 (deployed, field-validated)

Figure 3.10: v1 RF_HAT : 6-layer Pi HAT containing the dual-CC1200 tranceivers.

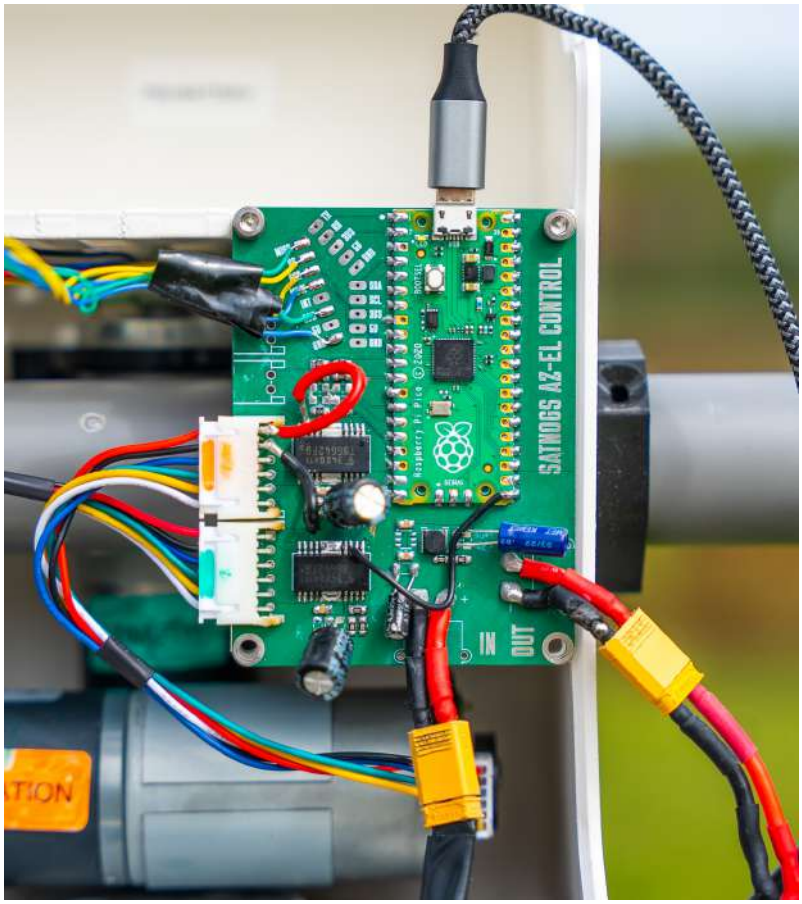

Figure 3.11: v1 MotorPCB assembled prototype, mounted in the rotator electronics bay.

Component	€
MotorPCB (KiCad 9, 4-layer, 13 sub-sheets, JLCPCB assembled)	∼30
RF HAT (6-layer Pi HAT, 2× CC1200, JLCPCB assembled)	∼55
Raspberry Pi 3 Model A+	25
Mechanical (motors, ASA, bearings, hardware)	∼35
v1 total	**∼€145**

Table 3.3 v1 (as-deployed) cost breakdown.

3.6.2 v2 (MOTOR_RF_HAT, design-complete, ready to order)

The v2 board merges motor control and RF front-end onto a single 6-layer Pi HAT. The A4950 (SOIC-8-EP) replaces the TB6642FG (HSOP-16) for availability (the TB6642FG went out of stock at JLCPCB during the project), and four 100 Ω series resistors (R11–

R14) on the GPIO-to-driver lines limit transients during bootloader cycling. One RP2040 handles both motor control and RF, so the USB cable and separate 5 V path are removed. Hand assembly avoids JLCPCB's extended-part setup fee (∼€18/board for the many precise RF L/C values).

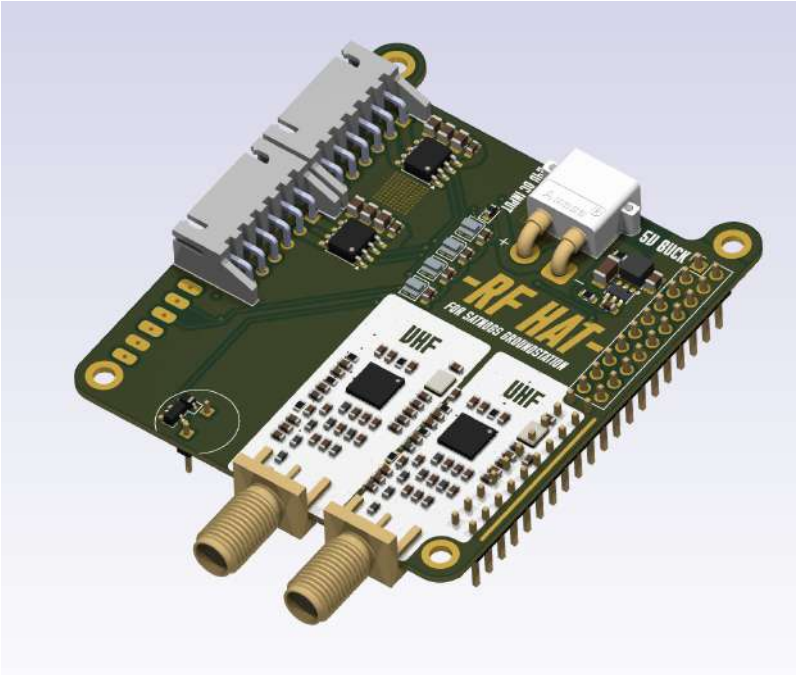

Figure 3.12: v2 MOTOR_RF_HAT (front): 6-layer Pi HAT merging motor control and dual-CC1200 RF.

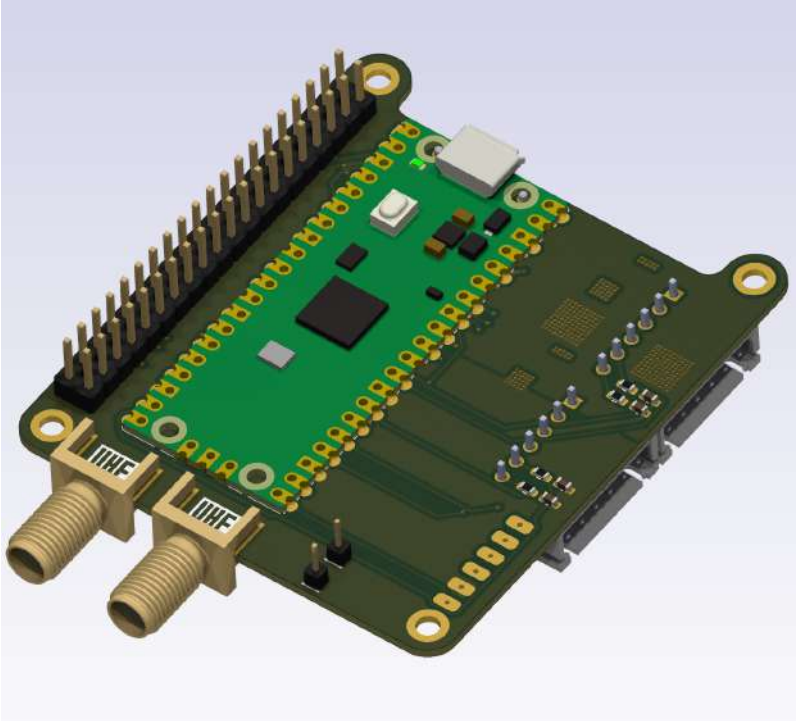

Figure 3.13: v2 MOTOR_RF_HAT (back).

Component	€
MOTOR_RF_HAT PCB	
Components (LCSC, incl. MOQ rounding)	28.44
SMA connectors + 15 nH inductors (not on LCSC)	2.00
Raspberry Pi Pico (hand-soldered)	4.00
Bare PCB (6-layer, JLCPCB, per board from 5-pc order)	6.00
PCB subtotal	**∼40**
Off-board	
FAPG36-555-EN motors (2×)	16.00
ASA filament (∼400 g)	10.00
Steel balls, screws, heat-set inserts	5.00
PVC pipe + misc hardware	2.00
ADXL345 IMU breakout	1.50
Mechanical subtotal	**∼35**
Station computer	
Raspberry Pi 3 Model A+	25.00
v2 station-core total	**∼€100**

Table 3.4 v2 (MOTOR_RF_HAT, projected) cost breakdown. Excludes antenna, coax, power supply, and tripod.

Excludes antenna, coax, power supply, and tripod (user-supplied). v2 brings the rotator BOM (PCB + motors + mechanics, excluding the Pi) to ∼€75, or ∼€40 for the combined controller/RF PCB alone. This misses a strict full-rotator €50 target but stays well below existing open-source and commercial rotators (Table 2.2).

Chapter 4

RF Front-End

The dual CC1200 hardware-modem approach adopted here departs from the canonical SatNOGS SDR receive chain. The rationale for that choice and the trade-offs against an RTL-SDR alternative are stated in Chapter 1 (§ *A deliberate alternative, not a replacement*); in brief, the CC1200 matches the AetherSpace on-board transceiver directly, exercises a complete embedded RF stack as educational scope, and fits within the Raspberry Pi 3A+ memory budget where the SatNOGS Docker stack would not. Chapter 6 quantifies the sensitivity cost of this choice and identifies the external LNA required to close the link. This chapter details the resulting RF hardware across three design iterations.

4.1 RF Design Progression

The RF front-end was developed across three hardware iterations. Each iteration eliminated a specific risk before committing to the next level of integration.

Table 4.1 RF hardware iteration summary.

	Devboard	**v1 RF HAT**	**v2 MOTOR_RF_HAT**
PCB layers	4	6	6
Form factor	Standalone	Raspberry Pi HAT	Raspberry Pi HAT
CC1200 count	1 (UHF only)	2 (UHF + VHF routed)	2
RF routing	CPWG	Microstrip	Microstrip
Host interface	USB CDC	UART	UART
Motor control	No	No	Yes
Status	Bench-validated	Field-deployed	Design-complete

The devboard was the first physical instantiation of the CC1200 transceiver stack. Its purpose was validation rather than integration: confirming that the SPI driver, SmartRF

register loading, COBS binary protocol, and bidirectional TX/RX FIFO paths all function correctly on real silicon, before any HAT-specific constraints were imposed. Section 4.2 describes the devboard architecture and presents the bench test results.

Once the devboard confirmed the RF stack, the v1 RF HAT translated the design to the Raspberry Pi HAT form factor. The transition introduced four changes: the 4-layer stackup became 6 layers to achieve controlled-impedance routing within the HAT footprint; a second CC1200 was added for a VHF uplink path; the USB CDC host interface was replaced by UART to connect directly to the Pi's GPIO header; and the devboard's dedicated power chain (24 V buck + dual 3.3 V LDOs) was replaced by the Pico module's onboard regulator to reduce cost and component count. The CC1200 driver, matching-network topology (§4.5.2), and COBS protocol were carried over unchanged.

The v2 MOTOR_RF_HAT merges the motor controller and RF front-end onto a single Pi HAT. The RF sub-circuit, matching networks, and SMA connectors are identical to v1. Changes are limited to the board-level integration: one RP2040 replaces the two separate Pico modules, and the A4950 motor driver replaces the discontinued TB6642FG. Section 4.6 summarises the RF-relevant differences; Chapter 3 covers the mechanical and cost implications.

4.2 RF Devboard

4.2.1 Architecture

The RF devboard is a 4-layer standalone PCB implementing a single CC1200 UHF transceiver controlled by an RP2040 microcontroller. It connects to a host PC over USB CDC using a COBS-framed binary protocol, and exposes a single SMA antenna connector. There is no HAT connector and no VHF path. The power design is self-contained: a 24 V input is stepped down to 5 V by a buck converter, followed by two separate 3.3 V LDOs, one supplying the digital logic and one dedicated to the CC1200. The v1 HAT simplified this by relying on the Pico module's onboard RT6150 regulator, trading supply isolation for lower component count and cost.

The devboard firmware and USB CDC binary protocol are described in Chapter 5 (§5.6).

The RF traces on the devboard use grounded coplanar waveguide (CPWG) routing. The 4-layer stackup and the absence of HAT form-factor constraints left sufficient board area for the lateral ground planes and via fencing that CPWG requires, making it the natural choice for a prototype board where RF performance takes priority over density. The matching network topology is the same three-path LC network used in the v1 HAT (§4.5.2), with component values from the TI CC1200EM 420–470 MHz reference design [27].

Figure 4.1: Two RF Devboards connected to usb

4.3 RF HAT Electrical Design

This section covers the electrical design of the v1 RF HAT. The changes from the devboard are summarised in §4.1; layout considerations specific to the 6-layer stackup are treated in the following section.

4.3.1 System Overview

The RF HAT is based on a microcontroller-controlled dual-transceiver architecture supporting both the VHF (144 MHz) and UHF (433 MHz) frequency bands.

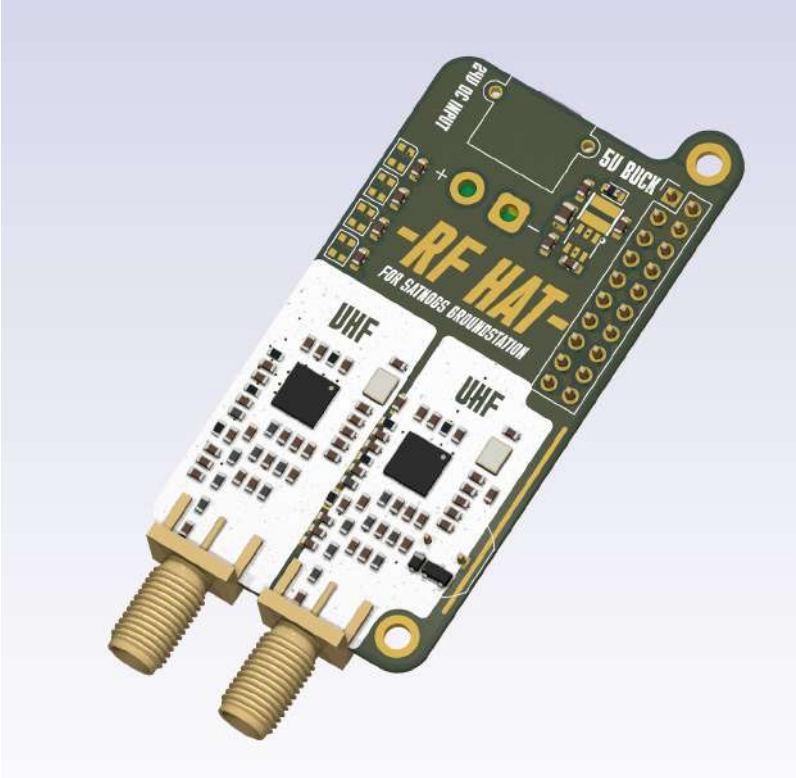

Figure 4.2: KiCad 3D render of the v1 RF Hat front

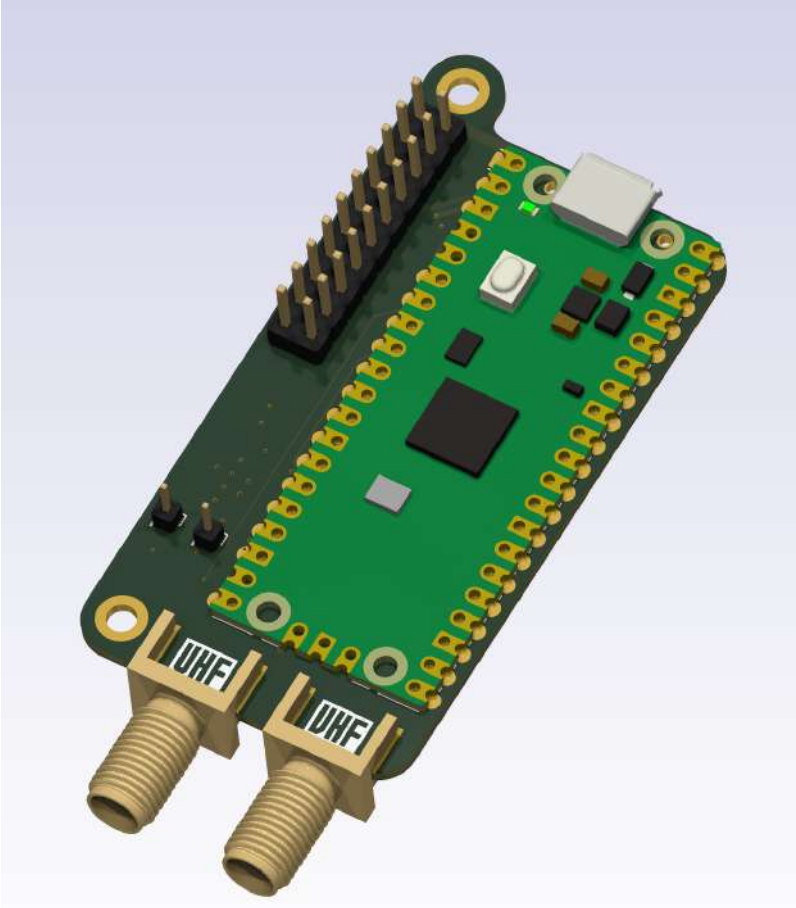

Figure 4.3: KiCad 3D render of the v1 Hat back

The design consists of the following functional blocks:

- An RP2040-based microcontroller for system control and data handling
- Two RF transceivers (VHF and UHF)
- Dedicated SPI communication interfaces
- A UART interface to the Raspberry Pi host
- Power distribution

4.3.2 Microcontroller Subsystem

The v1 RF HAT is controlled by a Raspberry Pi Pico microcontroller based on the RP2040 [28]. This device acts as the central control unit of the RF system.

Its responsibilities include:

- Configuration of both RF transceivers using SPI

- Handling of transmit and receive data
- Communication with the host system via UART
- Control of auxiliary signals such as chip select and interrupt lines

The RP2040 was selected due to its dual-core architecture and availability of two independent hardware SPI peripherals, enabling independent communication with both RF transceivers without bus arbitration [28]. Compared to typical single-SPI microcontrollers, this reduces firmware complexity and improves timing determinism.

Furthermore, the platform supports rapid firmware development and iteration through a well-established software ecosystem, enabling efficient testing and debugging during the development process.

Table 4.2 RP2040 Pin Assignment for v1 RF HAT

GPIO	Function	Connected To
GPIO0	UART TX	Raspberry Pi RX (J3)
GPIO1	UART RX	Raspberry Pi TX (J3)
GPIO10	SPI (UHF) SCLK	CC1200 UHF SCLK
GPIO11	SPI (UHF) MOSI	CC1200 UHF SI
GPIO12	SPI (UHF) MISO	CC1200 UHF SO
GPIO13	SPI (UHF) CS	CC1200 UHF ~CS
GPIO8	UHF reset	CC1200 UHF ~RESET
GPIO7	UHF GPIO0	CC1200 UHF GPIO0
GPIO9	UHF GPIO2	CC1200 UHF GPIO2
GPIO6	UHF GPIO3	CC1200 UHF GPIO3
GPIO18	SPI (VHF) SCLK	CC1200 VHF SCLK
GPIO19	SPI (VHF) MOSI	CC1200 VHF SI
GPIO16	SPI (VHF) MISO	CC1200 VHF SO
GPIO17	SPI (VHF) CS	CC1200 VHF ~CS
GPIO20	VHF reset	CC1200 VHF ~RESET
GPIO22	VHF GPIO0	CC1200 VHF GPIO0
GPIO21	VHF GPIO2	CC1200 VHF GPIO2
GPIO26	VHF GPIO3	CC1200 VHF GPIO3

4.3.3 RF Transceiver Subsystem

The RF HAT employs two CC1200 transceivers to enable dual-band operation [7]. Each transceiver can operate independently and includes its own supporting circuitry:

- A 40 MHz crystal oscillator
- External matching and filtering electronics
- Decoupling capacitors for supply stabilization

The use of two dedicated transceivers eliminates the need for an external band-select relay between UHF and VHF, simplifying the signal path and avoiding additional insertion losses.

The detailed layout and matching-network design of the RF front-end are discussed in the following sections.

4.3.4 Communication Interfaces

4.3.4.1 SPI Communication

Each RF transceiver is connected to the microcontroller via a dedicated hardware SPI interface on the RP2040.

The SPI interfaces consist of the following signals:

- Serial clock (SCLK)
- Master-out slave-in (MOSI)
- Master-in slave-out (MISO)
- Chip select (CS)

Interrupt lines from each transceiver are connected to the microcontroller to signal events such as packet reception or transmission completion.

Using separate SPI buses avoids contention and ensures deterministic communication timing.

4.3.4.2 UART Interface to Host

Communication between the RF HAT and the Raspberry Pi host is implemented via a UART interface.

This interface is used for:

- Transferring received RF data to the host
- Receiving configuration and control commands
- Debugging and diagnostic output

4.3.5 Power Distribution Network

The RF HAT is powered from a 24 V supply via an XT30 connector. This input voltage is converted to a regulated 5 V rail using an LMR51420 buck converter [29]. The resulting 5 V supply is used both to power the Raspberry Pi host through the board header and to supply the VSYS input of the Raspberry Pi Pico microcontroller.

Using a single 5 V rail for both the host and the onboard controller simplifies the overall power architecture and reduces the required component count.

The Raspberry Pi Pico generates the required 3.3 V rail using its onboard RT6150 switching regulator [30]. This 3.3 V supply powers both CC1200 transceivers as well as the associated digital circuitry on the RF PCB. While switching regulators typically introduce more noise compared to dedicated low-noise linear regulators, the integrated solution provides adequate current capability and efficiency for the system.

Given the noise sensitivity of RF circuits, particular attention was paid to supply integrity. Local decoupling capacitors were placed close to the supply pins of each active component to reduce high-frequency noise, provide transient current during switching events, and improve overall system stability. In addition, careful PCB layout practices were applied to minimize coupling of digital switching noise into the RF signal paths.

4.3.5.1 Power Supply Noise Evaluation

Supply integrity was assessed qualitatively during bench operation. No voltage drops or disturbances were observed that could affect transceiver stability, and the system demonstrated reliable communication at all tested configurations, confirming that the supply quality is sufficient for the intended application.

These results scope the 3.3 V supply rail at the CC1200 pins to within the transceiver's supply-ripple requirements. In-band RF noise at the antenna (characterised in §6.5) is a separate concern driven by board-level radiated/conducted interference, not by supply-rail quality; the principal mitigation is a mast-mounted LNA (§7.3). For future iterations, a dedicated low-noise LDO on the RF section may be considered to further improve supply integrity.

4.3.6 Summary

The RF HAT electrical design integrates a microcontroller, dual RF transceivers, and dedicated communication interfaces into a modular and robust architecture. This design forms the foundation for the layout considerations discussed in the following section.

4.4 RF HAT PCB Layout

The performance of the RF front-end is strongly influenced by the PCB layout. While both the 144 MHz (VHF) and 433 MHz (UHF) bands are sub-GHz and thus relatively low in frequency compared to microwave systems, careful RF design is still required to ensure impedance control, minimize losses, and reduce electromagnetic interference. The devboard (§4.2) used a 4-layer stackup without compactness constraints; the v1 RF HAT requires a 6-layer stackup to fit within the Raspberry Pi HAT form factor while maintaining controlled-impedance routing.

4.4.1 6-Layer Stackup Strategy

The PCB is implemented using JLCPCB's `JLC06121H-3313` 6-layer stackup with a finished thickness of 1.14 mm [31].

Table 4.3 6-layer PCB stackup (JLC06121H-3313) with electrical function and material properties [31].

Layer	Type	Thickness	Material	ε_r	Function
F.Cu	Copper	0.035 mm (1 oz)	-	-	RF routing, matching networks, SMA feedlines
Dielectric 1	Prepreg 3313	0.0994 mm	NP-155F	4.1	
In1.Cu	Copper	0.0152 mm (0.5 oz)	-	-	Primary ground plane (RF return path)
Dielectric 2	Core	0.35 mm	NP-155F	4.36	
In2.Cu	Copper	0.0152 mm (0.5 oz)	-	-	Power distribution (3.3 V, 5 V)
Dielectric 3	Prepreg 2116	0.1088 mm	NP-155F	4.16	
In3.Cu	Copper	0.0152 mm (0.5 oz)	-	-	Secondary ground plane (coupling reduction)
Dielectric 4	Core	0.35 mm	NP-155F	4.36	
In4.Cu	Copper	0.0152 mm (0.5 oz)	-	-	Digital and control signal routing
Dielectric 5	Prepreg 3313	0.0994 mm	NP-155F	4.1	
B.Cu	Copper	0.035 mm (1 oz)	-	-	Auxiliary signals and component placement

This stackup ensures that RF traces on the top layer (F.Cu) are implemented as controlled-impedance microstrip lines referenced to the continuous ground plane on In1.Cu. This keeps the current return path and loop area minimal, and prevents additional radiation and impedance discontinuities. The use of two ground planes (In1.Cu and In3.Cu) improves electromagnetic shielding and reduces coupling between RF and digital layers. Additionally, separating digital routing on In4.Cu minimizes interference with sensitive RF signals on the top layer.

4.4.2 Transmission Line Implementation

All RF signal paths are routed with controlled impedance considerations, targeting a characteristic impedance of 50 Ω, which is standard for RF systems and ensures proper matching with external components.

The characteristic impedance of a microstrip line can be approximated using the narrow-strip Hammerstad expression [32]:

$$Z_0 = \frac{60}{\sqrt{\varepsilon_{\text{eff}}}} \ln\left(\frac{8h}{w} + \frac{w}{4h}\right) \tag{4.1}$$

with the effective dielectric constant given by:

$$\varepsilon_{\text{eff}} = \frac{\varepsilon_r + 1}{2} + \frac{\varepsilon_r - 1}{2}\left(\frac{1}{\sqrt{1 + 12h/w}}\right) \tag{4.2}$$

where w is the trace width, h is the dielectric height between the trace and the ground plane, and ε_r is the relative permittivity of the substrate. This form is strictly valid for $w/h < 1$; for wider traces a modified expression applies, though the two forms converge near the transition point and their results agree to within 10–15% at $w/h \approx 1.5$.

Applying these expressions with the selected stackup parameters ($h = 0.0994\,\text{mm}$, $\varepsilon_r = 4.1$) yields an initial estimate of $w \approx 0.20\,\text{mm}$, giving $w/h \approx 2.0$. This result falls in the wide-strip regime ($w/h > 1$), outside the strict validity domain of the narrow-strip form; the estimate is therefore used only as an order-of-magnitude check. Cross-validating with the JLCPCB impedance calculator [33], which implements a full numerical model including the finite copper thickness (1 oz, 0.035 mm) and is valid across all w/h ratios, yields $w = 0.1565\,\text{mm}$ ($w/h \approx 1.57$) for a $50\,\Omega$ single-ended non-coplanar microstrip on this stackup. The KiCad net class was set to 0.15 mm (6 mil), the nearest standard routing increment, corresponding to approximately 52–53 Ω, which is within accepted tolerance for the operating frequencies.

At the operating frequencies, the free-space wavelengths are approximately 2.08 m at 144 MHz and 0.69 m at 433 MHz. Since the physical dimensions of the matching network (approximately 10 mm $\times$ 15 mm) remain well below $\lambda/10$, the interconnects are electrically short and distributed effects are limited.

The RF devboard used grounded coplanar waveguide (CPWG) routing. CPWG confines the electromagnetic field between the signal trace and its flanking ground conductors, reducing radiation and improving isolation from surrounding structures; the devboard's 4-layer standalone form factor provided sufficient board area for the required lateral clearances and via fence. The v1 HAT transitions to microstrip: the HAT footprint does not allow the lateral ground planes and via fencing that CPWG demands, and the continuous In1.Cu ground plane immediately below F.Cu provides a well-defined return path that makes microstrip a controlled and predictable transmission-line medium at these frequencies.

By maintaining a continuous ground reference, minimising trace lengths, and applying sufficient ground stitching, the performance degradation relative to CPWG is expected to remain limited.

4.4.3 Frequency-Dependent Layout Considerations

Although both frequency bands share the same PCB layout strategy, their sensitivity to layout effects differ.

At 144 MHz, the wavelength is relatively large, and PCB interconnects behave predominantly as lumped elements. As a result, the layout is relatively tolerant to small discontinuities and parasitic effects.

At 433 MHz, however, the shorter wavelength makes PCB parasitics more significant. RF traces must therefore be treated as transmission lines, requiring controlled impedance routing, short interconnects, and dense via stitching to maintain a stable ground reference.

To ensure reliable operation across both bands, the PCB layout is designed according to the more stringent requirements of the 433 MHz band. This guarantees that performance at 144 MHz is inherently satisfied.

4.4.4 Summary

A 6-layer PCB stackup was selected to enable a compact RF front-end while maintaining controlled impedance routing and effective isolation between RF, power, and digital domains. The layout is designed according to the more stringent requirements of the 433 MHz band, ensuring reliable operation across both frequency bands within the given form factor constraints.

4.5 RF Matching Network

4.5.1 CC1200 RF Interface Characteristics

To properly design the matching network, it is necessary to consider the RF interface characteristics of the CC1200 as specified in the datasheet [7].

4.5.1.1 Operating Frequency Bands

There are TI designs for operation in the following frequency ranges:

- 164–190 MHz
- 410–475 MHz
- 820–960 MHz

These bands correspond to typical ISM and SRD applications. Operation outside these ranges may be possible but requires careful validation. [7]

4.5.1.2 CC1200 RF IO Structure

The receiver input of the CC1200 consists of a differential low-noise amplifier (LNA) with input pins LNA_P and LNA_N. These pins require a DC path to ground and are intended to be driven by a differential RF signal [7].

The transmitter output is single-ended at the PA pin and requires a DC path to the supply voltage.

In addition, the CC1200 provides a TRX_SW output pin that the chip drives high during transmission and low during reception [7]. The external matching network uses this switching signal to route signals between the PA output, the LNA input, and the shared antenna port. During transmission, TRX_SW couples the PA output through the TX matching path to the SMA connector; during reception, it decouples the PA and connects the antenna signal through a coupling network to the differential LNA inputs. This mechanism allows a single SMA connector per transceiver to serve both transmit and receive without an external RF switch.

4.5.1.3 Optimum Source Impedance (Receiver)

The CC1200 does not present a fixed 50 Ω input impedance. Instead, the optimal source impedance is frequency-dependent and complex. The datasheet provides reference values for the two bands most relevant to this design [7]:

- 433 MHz band:
  - Differential: $100 + j60\,\Omega$
  - Single-ended equivalent: $50 + j30\,\Omega$
- 169 MHz band (closest datasheet entry to the 144 MHz design target):
  - Differential: $140 + j40\,\Omega$
  - Single-ended equivalent: $70 + j20\,\Omega$

No datasheet entry exists for 144 MHz. The 169 MHz values are used as the nearest available reference; the 144 MHz matching network was designed by scaling component values accordingly (see Section 4.5.2).

These values represent the optimum impedance for maximum receiver sensitivity and must be realized through the external matching network.

4.5.1.4 Optimum Load Impedance (Transmitter)

Similarly, the transmitter output requires impedance matching to a complex load. The datasheet specifies [7]:

- 433 MHz band: $55 + j25\,\Omega$
- 169 MHz band (closest reference to the 144 MHz design target): $80 + j0\,\Omega$

This further emphasizes that the RF interface is not inherently matched to $50\,\Omega$, and external matching is mandatory.

4.5.1.5 Receiver Sensitivity and Matching Impact

The CC1200 achieves high sensitivity, for example:

- Up to -123 dBm at 1.2 kbps 2-FSK with 4 kHz deviation and 11 kHz channel filter bandwidth in the 433 MHz band

This performance is specified under optimal matching conditions at the antenna port. Any mismatch between the antenna and the IC will degrade sensitivity due to reduced power transfer and increased noise figure. [7]

4.5.1.6 Output Power and Power Consumption

The transmitter is capable of delivering [7]:

- Up to +16 dBm at 433 MHz with a 3.3 V supply rail
- Programmable output power with 0.4 dB step size

At this output level, the current consumption is approximately:

- 46–49 mA in high-performance mode at 433 MHz

These values highlight the importance of efficient matching to avoid unnecessary power loss.

4.5.1.7 Implications for Matching Network Design

The combination of:

- Complex, frequency-dependent RF impedance
- Differential receiver input
- Single-ended transmitter output
- Shared antenna port for TX and RX via `TRX_SW`

necessitates the use of a multi-element matching network that performs impedance transformation, single-ended to differential conversion, and TX/RX path routing simultaneously.

This justifies the use of the three-path LC topology implemented in this work, rather than a simple L-section network.

4.5.2 Matching Network Topology

4.5.2.1 433 MHz Front-End (Validated Design)

The implemented 433 MHz matching network is realized between SMA connector J1 and CC1200 transceiver U1, based on the TI CC1200EM reference design for the 420–470 MHz band [27] and the M17 CC1200_HAT Rev B hardware [34]. The network is organized across three functional paths:

- **TX matching path**: series inductors L1 (15 nH), L2 (43 nH), and L3 (22 nH) transform the PA output impedance to 50 Ω, with feedback capacitor C1 (2.2 pF), series capacitor C2 (39 pF), shunt capacitor C4 (6.2 pF), and DC-blocking capacitor C10 (1 nF) at the SMA interface.
- **PA supply bias path**: RF choke L4 (56 nH) and bypass capacitor C5 (56 pF) provide the required DC supply path to the `PA` pin while presenting high impedance to RF signals.
- **TRX coupling and LNA balun**: inductor L5 (15 nH) and capacitor C3 (5.1 pF) form the `TRX_SW`-driven coupling network. The differential LNA input is driven through bridge inductor L7 (56 nH) connecting `LNA_P` and `LNA_N`, with bias inductors L6 and L8 (27 nH each) and coupling/shunt capacitors C8 and C9 (5.1 pF each).

This multi-element structure provides gradual impedance transformation, improved bandwidth stability, and additional harmonic filtering compared to a simple L-section network.

4.5.2.2 144 MHz Front-End (Optional, Not Populated)

In addition to the validated 433 MHz front-end, the PCB includes an optional 144 MHz RF path between SMA connector J2 and transceiver U2. The component values were derived by adapting the topology of the TI CC1120EM 169 MHz reference design [35] and scaling for operation at 144 MHz. As no official TI reference design exists for 136–160 MHz, inductor values were scaled by approximately 17 % relative to the 169 MHz design. The three-path network uses inductors L9–L16 (U2 instance). Per the KiCad netlist: L14 (22 nH, TX match), L15 (120 nH, TX match), L16 (82 nH, TX match), L13 (270 nH, PA choke), L11 (47 nH, TRX coupling), and L9, L10, L12 (100 nH each, LNA

balun elements). Capacitors are numbered in the C11 and C20–C47 range in the global KiCad annotation; the exact designator-to-function mapping should be confirmed against the project netlist before final assembly.

It is important to note that this 144 MHz front-end was included as an additional design option but was **not populated on the final PCB**. Consequently, it was **not experimentally validated**, and its performance has not been verified in practice.

4.5.3 Signal Transformation Through the Matching Network

The matching network simultaneously serves three functional paths that share the single SMA antenna port.

4.5.3.1 TX Path: PA Output to SMA

During transmission, the CC1200 drives the single-ended PA pin. The TX matching path performs impedance transformation from the PA output impedance ($55 + j25\,\Omega$ at 433 MHz) to the $50\,\Omega$ antenna interface. A DC-blocking capacitor at the SMA side prevents the PA supply bias from reaching the antenna, while the series LC sections move the impedance along the Smith chart toward the matched condition. Distributing the transformation across multiple elements reduces sensitivity to component tolerances and insertion loss.

4.5.3.2 TRX Switching Path

The TRX_SW pin output drives a coupling network (L5, C3) that connects the TX and RX paths to the shared antenna node. During transmission, TRX_SW is high and the coupling network routes the PA signal to the antenna. During reception, it is low, isolating the PA and directing the antenna signal toward the LNA balun. This arrangement avoids the insertion loss of an external RF switch.

4.5.3.3 RX Path: SMA to Differential LNA Input

During reception, the single-ended $50\,\Omega$ antenna signal must be converted to the differential input required by the LNA. The LNA balun consists of a bridge inductor (L7) connecting LNA_P and LNA_N, with separate bias inductors (L6, L8) providing the required DC path to ground and coupling elements to the TRX node. This quasi-differential arrangement splits the single-ended signal into two anti-phase components, effectively performing a passive balun function. The component values are chosen so that the impedance presented to the LNA inputs matches the optimum source impedance ($100 + j60\,\Omega$ differential at 433 MHz) for maximum sensitivity.

4.5.4 Frequency-Selective Behavior

In addition to impedance matching, the network also exhibits low-pass filtering characteristics. This is particularly important at UHF frequencies such as 433 MHz, where harmonic emissions can affect system performance and regulatory compliance.

The cascaded LC structure attenuates higher-order harmonics, improving spectral purity.

4.5.5 Use of the Reference Design

The design of the 433 MHz matching network is based on the Texas Instruments CC1200EM reference design for the 420–470 MHz band [27]. Component values were further cross-referenced against the M17 CC1200_HAT Rev B, an open-hardware design verified at +14 dBm output at 433 MHz [34]. These references provide a validated topology and component selection strategy.

The 144 MHz matching network was adapted from the TI CC1120EM 169 MHz reference design [35], which is the closest available TI reference for the VHF band.

In this work, the reference designs are used as a guideline for:

- Matching network topology
- Impedance transformation strategy
- Differential interface implementation

The final schematic and layout are specific to this project and adapted to the chosen PCB stackup and form factor.

4.5.6 Summary

The RF matching network is a critical part of the front-end design, enabling efficient signal transfer between the 50 Ω antenna interface and the differential RF input of the CC1200.

The validated 433 MHz implementation uses a three-path LC network (TX match, PA bias, TRX coupling and LNA balun) derived from the TI CC1200EM and M17 CC1200_HAT reference designs and adapted to the project-specific PCB. An additional 144 MHz front-end was included as an optional extension but was not populated or experimentally verified.

4.6 V2 RF Sub-Circuit

The v2 MOTOR_RF_HAT integrates the RF front-end and the antenna rotator motor controller onto a single Raspberry Pi HAT. The RF sub-circuit is carried over from v1 unchanged: the same dual CC1200 arrangement, matching networks, SMA connectors, and SmartRF register profiles are used. No RF redesign was performed.

Changes relative to v1 are limited to board-level integration. The two Pico modules are replaced by a single RP2040 mounted directly on the board. The TB6642FG motor driver, which was discontinued, is replaced by the A4950. GPIO lines connecting the RP2040 to the A4950 inputs are routed through 100 Ω series resistors to limit switching transients. The combined board reduces cabling and connector count while keeping the RF performance identical to the validated v1 design. The v2 board was fully routed but has not been assembled or tested at the time of writing.

Chapter 5

Firmware and Software

5.1 Rotator Firmware: PID Control

The closed-loop controller is a discrete-time PID at 100 Hz, executed from the main `for(;;)` loop on core 0 (period-gated with `absolute_time_diff_us`, not a hardware timer). Gains and constants are set in `Firmware/rp2040-satnogs-rotator/src/config.h`:

- $K_p = 0.15$, $K_i = 0.03$, $K_d = 0.02$
- Deadband: 0.05° (error below which no correction is applied)
- Sample period: $\Delta t = 0.01$ s

Implementation details:

- Anti-windup. Integral accumulation is skipped when $|u| \geq 0.95$ and the error sign would grow the integral further.
- Derivative filter. First-order LPF on the derivative term, $\alpha = 0.15$, suppresses encoder-quantisation noise at the 0.004°–0.008° resolution from §3.2.3.
- Duty output filter. First-order LPF on the PWM duty, $\alpha = 0.05$ initially; raised to $\alpha = 0.15$ during field testing (§6.2).
- Maximum duty: 0.95, to avoid TB6642FG overcurrent on stall.

Both axes share the same PID. The elevation axis sets `kElInvert = true` in `config.h`, which inverts both the encoder reading and the PID output sign. Inverting only one of the two produces the positive-feedback runaway documented in §3.5.

Step-response plots and settling-detail are quantified in Chapter 6 (§6.2).

5.2 Quadrature Encoding

The Hall-effect encoder on each motor shaft gives two channels A, B. 4× decode produces the 64 CPR / 244.6 AZ ticks/deg / 126.1 EL ticks/deg resolution derived in §3.2.3.

The decoder is a GPIO-IRQ-driven Gray-code state machine running on core 0. At the 35°/s peak slew (§6.1), the inter-edge period on the fastest axis is $1/(35 \cdot 244.6) = 117\ \mu$s, which is comfortably above the per-edge ISR service time.

5.3 Homing and Calibration

EL: ADXL345 gravity vector → drive to horizontal → zero encoder. The `RECAL` command re-homes during operation.

AZ: assume north at boot. User points the station north (guided by a compass, conveniently the phone compass when browsing the dashboard on a phone) and confirms via the setup wizard; the `ZERO` command sets the current position as 0°.

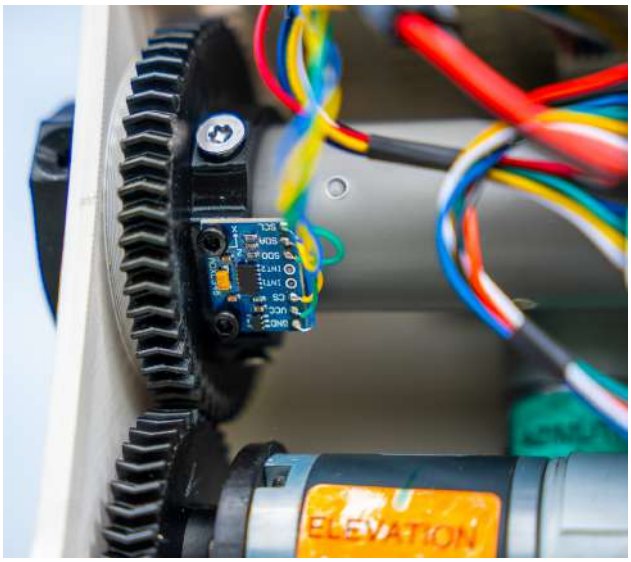

Figure 5.1: ADXL345 accelerometer breakout mounted on the elevation frame for EL homing.

5.4 Safety

- Runaway detection: e-stop if position exceeds soft limits by 50°.
- Shortest-path wrapping: 350°→ 10° takes a 20° path rather than 340°; re-centres the encoder after settling near ±360°.
- 8 s hardware watchdog enabled after EL homing completes. Homing is protected by its own 15 s software timeout.

5.5 EasyComm Implementation

Standard commands: AZ/EL set/get, PARK, STOP, VE, GS/GE. Custom extensions: RESET, RECAL, MONITOR, TICKS, ZERO, GAINS, IMU.

Custom commands are invisible to hamlib, so compatibility with rotctld, GPredict, and the satnogs-client rotator path is maintained. The full EasyComm + extensions parser is implemented in satnogs_protocol.cpp (∼250 lines of C++).

5.6 RF Devboard Firmware

The devboard firmware runs on the RP2040 and communicates with a host PC over USB CDC. The transport uses the same COBS framing and CRC-16/CCITT-FALSE as the later RF HAT firmware, but over USB instead of UART. The RP2040 drives the CC1200 over SPI0 at 5 MHz; on startup it resets the transceiver and reads the part identification registers to confirm communication before entering the protocol polling loop.

The runtime SmartRF register profile loading mechanism, the three-phase PROFILE_BEGIN / PROFILE_CHUNK / PROFILE_APPLY sequence that writes all register values in a single atomic batch with the radio held in IDLE, was designed and validated on the devboard and carried over unchanged to the RF HAT firmware (§5.7).

5.7 RF HAT Firmware

Binary protocol: COBS framing + CRC-16/CCITT-FALSE over UART 115200.

Key message types:

- SELECT_RADIO (0x40): switch UHF/VHF CC1200.
- READ/WRITE_REG, READ/WRITE_EXT: CC1200 register access.
- PROFILE_BEGIN/CHUNK/APPLY: SmartRF profile loading (chunked, no reflash needed).
- SET_STREAMING: enable/disable continuous RX packet forwarding.
- SET_SERIAL_MODE (0x35): enable PIO-based serial bitstream capture.
- TX_WRITE/SEND: load FIFO + transmit.
- BUZZER (0x50): trigger patterns (ready, AOS, LOS, packet, error).
- PING: connectivity test; response is PING | 0x80 (0x81), not a separate type.
- EVT_RX_DATA (0xE0): asynchronous received data (firmware→Pi, both FIFO and serial modes).

CC1200 SPI driver: SPI1 = UHF (GP10–13), SPI0 = VHF (GP16–19), 5 MHz clock. Each on a separate SPI peripheral, so there is no bus contention.

4× WS2812B NeoPixels on GP4. Non-blocking buzzer tone sequencer on GP5 (hardware PWM via NPN/MOSFET gate).

5.7.1 CC1200 Serial Mode and AX.25 G3RUH Decoding

The CC1200's default FIFO mode requires sync-word detection before delivering data. G3RUH scrambling [36] randomises the bitstream before transmission, so the over-the-air bytes do not contain the pre-scrambling sync word and FIFO mode will either miss the frame or false-trigger on scrambled noise. Undocumented framing hits the same wall. The firmware therefore implements a synchronous serial receive mode using the RP2040's PIO (Programmable I/O) peripheral.

The CC1200 is reconfigured for synchronous serial output (`PKT_FORMAT=0x01`, `FIFO_EN=0`, `SYNC_MODE=0x00`) with GPIO0 carrying the demodulated bitstream and GPIO2 carrying the recovered bit clock. The internal demodulator runs continuously, outputting every demodulated bit.

A 3-instruction PIO program (`serial_rx.pio.h`) captures the bitstream into the CPU via autopush (Listing 5.1).

```
.wrap_target
    wait 1 pin 2    ; SERIAL_CLK rising edge
    in pins, 1      ; shift in SERIAL_RX
    wait 0 pin 2    ; SERIAL_CLK falling edge
.wrap
```

Listing 5.1: PIO program: clock-synchronous bit capture from the CC1200.

The PIO is gated by the CC1200 clock output, so no baud-rate setup is needed on the RP2040 side.

In blind mode the CC1200 outputs samples at approximately 15× the configured symbol rate (empirical; the firmware measures the ratio at startup as PIO word count over 100 ms divided by the expected symbol rate). An edge-triggered resampler then extracts data bits at the midpoint of each symbol period.

Two serial sub-modes are selected via the `MSG_SET_SERIAL_MODE` body byte:

1. Mode 1: raw serial. Demodulated bytes go to a 256-byte ring buffer, forwarded to the Pi as `EVT_RX_DATA` every 50 ms or 32 bytes. Used for blind capture of unknown framing.

2. Mode 2: AX.25 G3RUH decode. Each resampled bit is fed through a three-stage pipeline implemented in `ax25_decode.c`:

- G3RUH descrambler: 17-bit LFSR, polynomial $x^{17} + x^{12} + 1$, reverses the AMSAT scrambler.
- NRZ-I decoder: differential to absolute.
- HDLC deframer: 0x7E flag detection, bit-stuffing removal after five consecutive ones, LSB-first byte assembly, CRC-16/CCITT check (poly 0x8408, residual 0xF0B8). Minimum valid frame 17 bytes (14 address + 1 control + 2 FCS).

CRC-valid frames go up as `EVT_RX_DATA` with `type=0x01`; raw bytes are forwarded in parallel for diagnostics.

`station.py` selects serial mode automatically for satellites at $\geq$4800 bps FSK/GFSK (likely G3RUH), and FIFO mode with the AX.100 sync word 0x930B51DE for lower-rate satellites using the GOMspace NanoCom format.

5.8 Software Architecture and Data Flow

The Raspberry Pi software stack is organised as three cooperating layers that exchange data through well-defined boundaries: a real-time tracking daemon (`Pi/station.py`, $\sim$2950 LOC), a stateless HTTP/HTTPS gateway (`Pi/dashboard.py`, $\sim$2060 LOC), and a single-page browser application (`Pi/dashboard.html`, $\sim$5650 lines of HTML/CSS/JavaScript).

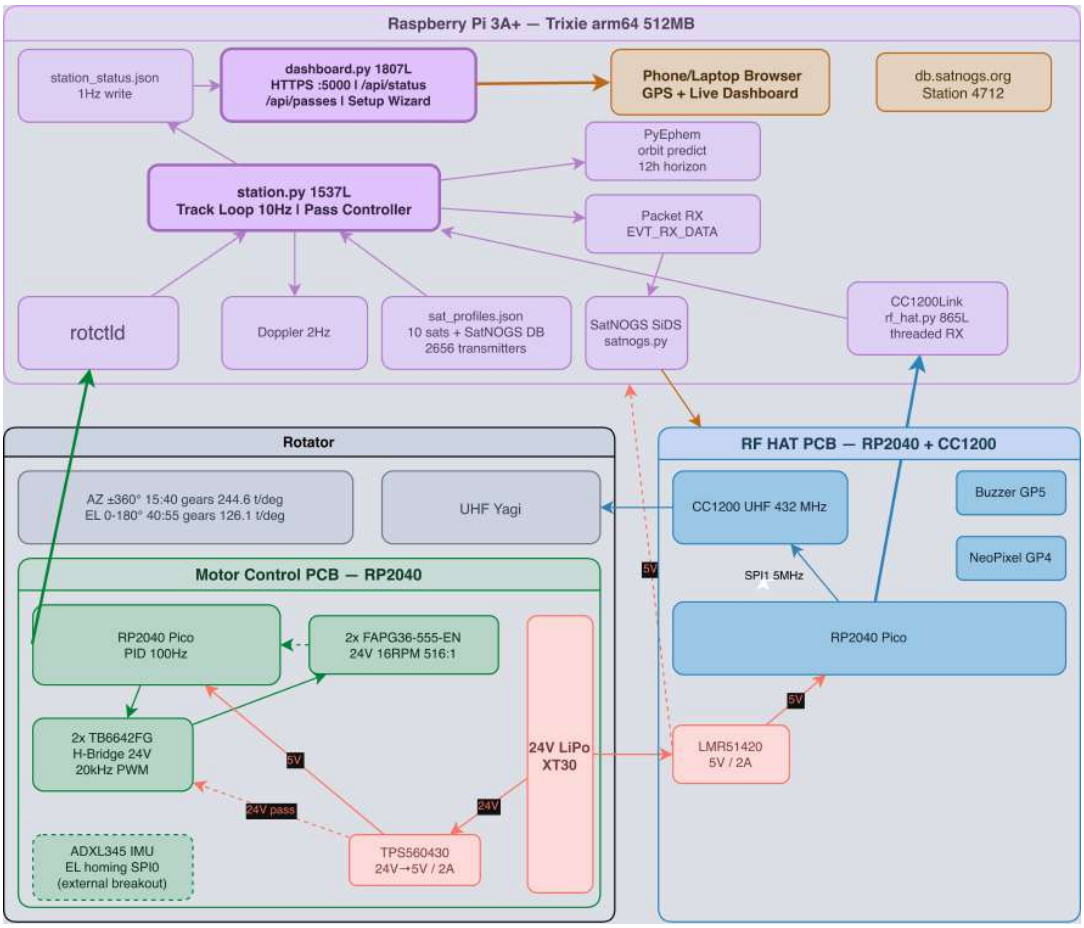

Figure 5.2: End-to-end data flow: external sources, Pi services, browser.

External data sources. Three external services feed the station: (i) the AMSAT bulletin `nasabare.txt` as primary TLE source; (ii) CelesTrak `GROUP=stations/amateur/weather/noaa` as fallbacks, plus per-NORAD CelesTrak queries for satellites listed in `sat_profiles.json` (typically MEO objects outside the AMSAT bulletin); and (iii) the SatNOGS DB transmitter API at `https://db.satnogs.org/api/transmitters/` for satellite frequency, modulation, and protocol metadata. Local cache files written under `~/` (`.station_tles.json`, `.station_status.json`, `.station_pass_history.json`) decouple the tracking loop from network availability.

Process boundaries. `station.py` owns the rotator (via the hamlib `rotctld` TCP socket on port 4533) and the RF HAT (via UART `/dev/serial0`). It writes a single atomic JSON snapshot, `~/.station_status.json`, every 200 ms during an active pass (5 Hz, gated by `tick % 2 == 0` at the 10 Hz tracking rate). `dashboard.py` runs in a separate process: it reads the same JSON every 500 ms (2 Hz internal poll), augments it with `rotctld` actuals, station configuration, and Pi system metrics, and exposes `/api/status`, `/api/passes`, `/api/pass-history`, `/api/track`, `/api/auto-track`, `/api/freq`, and a handful of control endpoints over HTTPS on port 5000. The browser polls `/api/status` every 500 ms

(2 Hz), /api/passes every 15 s, and /api/pass-history every 30 s. The status JSON is written via tempfile.mkstemp + os.rename, so the dashboard never observes a partial write.

Why separate the daemon from the dashboard. Two reasons. First, isolation: a browser disconnect, a TLS handshake stall, or a misbehaving HTTP client cannot pause the 10 Hz rotator command loop. Second, the dashboard exits cleanly during firmware updates while the daemon continues tracking. The price is a 200 ms write/read latency budget, which is well below the human-perceivable threshold for a status display.

5.9 TLE Sources, Pass Prediction, and Profile Selection

TLE acquisition

Two-line element sets are fetched in this order (Pi/station.py:353):

1. Read ~/.station_tles.json, accepted if its file modification time is within 7200 s (2 h). The dashboard maintains this file in its own background thread.
2. Otherwise, GET AMSAT's nasabare.txt (∼80–100 amateur satellites of interest).
3. Fall back to CelesTrak GROUP=amateur/stations/weather/noaa on AMSAT failure.
4. For any satellite listed in sat_profiles.json with a NORAD ID outside the AMSAT bulletin, issue a per-NORAD CelesTrak GET (?CATNR=).

The in-memory cache is gated by a 1800 s (30 min) TTL inside fetch_tles() so a single tracking process never re-issues network requests within half an hour. The on-disk cache (2 h TTL) survives Pi reboots, so a station that loses internet between passes still tracks until TLE accuracy degrades.

Pass prediction

dashboard.py owns pass computation. PyEphem propagates each TLE against an ephem.Observer positioned at the station's GPS coordinates from station.conf. The cache TTL is 300 s (5 min, PASS_CACHE_SECONDS) and recomputation runs in a background thread so HTTP requests never block. Each pass dictionary holds rise/set times, max-elevation azimuth, sampled trajectory, and the resolved per-satellite frequency, modulation, and protocol from the SatNOGS transmitter cache (TRANSMITTER_CACHE_SECONDS = 7200, 2 h). The station.py daemon does its own per-tick PyEphem computation at 10 Hz so the live trajectory advances continuously rather than in 5–10 s steps.

Per-satellite profile selection

Before a pass begins, `station.py` resolves a `SatProfile` for the target (`Pi/sat_library.py`) in three layers:

1. Local override. `Pi/sat_profiles.json` is consulted first, with a three-pass match priority: NORAD ID + name match, then name substring, then NORAD ID alone. Hand-curated entries pin a known-good frequency, modulation, symbol rate, deviation, RX bandwidth, sync word, and packet config. Approximately 30 satellites are pre-entered.
2. SatNOGS DB fallback. Unknown satellites trigger `lookup_with_satnogs_fallback()`, which scans the cached transmitter list, picks the best UHF transmitter (preferring active, non-CW, non-1200-baud-AFSK, CC1200-compatible modes), and fills in a profile via the `SATNOGS_MODE_MAP` dictionary that translates SatNOGS DB mode strings (e.g., "GMSK 9k6") into CC1200 modulation parameters.
3. Protocol inference. A second pass over the SatNOGS DB description string classifies the protocol: keywords `ax100`, `usp`, `ccsds`, `geoscan`, and `g3ruh` map to dedicated decoders; high-baud unknown modes are tentatively assumed to be G3RUH-scrambled AX.25 (marked `_assumed` in the status feed so the dashboard can colour them yellow rather than green).

The resolved profile is then applied to the CC1200 in one of three configuration tiers (`Pi/station.py:807`):

- Tier A (frequency-only). Keep the default 2.4 kbps 2-GFSK SmartRF profile and only retune the synthesizer. A handful of register writes per pass.
- Tier B (modulation override). Compute the ∼15 critical CC1200 registers (`MODCFG_DEV_E`, `DEVIATION_M`, `SYMBOL_RATE_*`, `CHAN_BW`, `SYNC_*`, `PKT_CFG*`, `PKT_LEN`) from the profile values and stream them via `WRITE_REG` messages.
- Tier C (full SmartRF). Stream a complete TI SmartRF Studio export via the `PROFILE_BEGIN/CHUNK/A` sequence of §5.7. Used for satellites whose modulation parameters do not fit Tier B (e.g., very low symbol rates, unusual deviation/bandwidth ratios). Chunked transfer is required because a full profile exceeds the COBS frame size.

The active tier is exposed in the status JSON as `rf.radio_config.config_tier` and rendered on the dashboard so the operator can verify which path was taken.

5.10 Doppler Correction Loop

The radial velocity of the satellite, v_r, is computed every 500 ms from the PyEphem solution (`DOPPLER_HZ = 2`, `Pi/station.py:214`); the Doppler-shifted frequency is

$$f_{rx} = f_0 \left(1 - \frac{v_r}{c}\right) \tag{5.1}$$

with $c = 299\,792\,458$ m/s (`Pi/rf_hat.py:359`). At $f_0 = 432$ MHz and a worst-case LEO radial velocity of ± 7.5 km/s the maximum Doppler is ± 10.8 kHz; the observed sweep across a high-elevation pass is ± 9 kHz, consistent with the radial-velocity flattening at high elevation. The implementation is shown in Listing 5.2.

```
def doppler_shift(freq_hz: float, range_rate_m_s: float) -> float:
    """range_rate > 0: receding (red shift); range_rate < 0:
       approaching."""
    c = 299_792_458.0
    return freq_hz * (1.0 - range_rate_m_s / c)
```

Listing 5.2: Doppler shift helper from `Pi/rf_hat.py`.

The CC1200 is retuned in two ways. The full `FREQ+FS_CFG` rewrite requires `SIDLE` (a few ms of receiver downtime) and is used at AOS, on tier change, or on a forced retune. Mid-pass updates are written to the 16-bit `FREQOFF` register, which adds an offset directly to the synthesizer without dropping out of RX. The same register, read back as `FREQOFF_EST`, is used as a residual-error display in the CC1200 debug panel: a non-zero estimate after Doppler correction indicates either a TLE error or a transmitter offset.

5.11 Status JSON Schema

The `~/.station_status.json` file is the single source of truth between `station.py` and the dashboard. The top-level keys reflect the layered architecture:

- `satellite`, `pass_progress`, `los_seconds`, `duration`, `cmd_az`, `cmd_el`, `rise_az`, `set_az`, `max_el`, `range_km`, `range_rate`, `doppler_hz`: per-tick orbital state from `station.py`.
- `rf`: `rssi` (dBm), `freq_mhz` (Doppler-corrected), `packets`, `streaming`, `rx_mode`, `protocol`, `radio_config` (Tier, sync, modulation, packet config), and a `cc1200` sub-object exposing low-level register state (`marc`, `flags`, `rssi_dbm_x10`, `freqoff_est_hz`, `lqi`, `rx_total_bytes`, `rx_overflow_count`, `rx_fifo_err_count`, `tx_fifo_err_count`, `uptime_ms`).
- `rotator`: `az`, `el`, `connected`, `state`, populated by `dashboard.py` from `rotctld` (the `p` command, every 500 ms).
- `auto_mode` (`off` / `track` / `scan`), `auto_track`, `rf_mode`, `scan_settings`: read from `station.conf` by `dashboard.py` and merged into the status payload.

- system: cpu_temp (/sys/class/thermal/thermal_zone0/temp), ram_used_mb, ram_total_mb (/proc/meminfo), filled in by dashboard.py.
- location: lat, lon, elev from station.conf.
- satnogs: enabled, station_id, submitted, queued, failed, last_submit from the SiDS submission helper.

5.12 Operating Modes: Off / Track / Scan

station.py runs in one of three modes, persisted as auto_mode in station.conf and switchable from the dashboard. The mode is re-read at every loop iteration so the operator's choice takes effect within one tracking tick.

- Off: rotator parked, RF idle. The daemon still refreshes TLEs and reports system metrics in the background, but it does not move the rotator or configure the CC1200. Used during transport and bench testing.
- Track: the daemon picks the next pass that exceeds the configured minimum elevation (default 5°, MIN_ELEV in Pi/station.py:215; the dashboard exposes 10° as the operator-facing default), pre-positions to AOS, then commands the rotator and CC1200 through the pass. After LOS it idles until the next visible pass.
- Scan: continuous multi-satellite cycling driven by run_scan_cycle() (Pi/station.py:2053). The daemon builds a build_schedule() of all currently-visible passes above scan_min_el (default 10°) for the next scan_hours (default 4 h) and dwells scan_dwell seconds (default 30 s) on each visit, then advances. The schedule is rebuilt on every cycle, so a freshly-risen satellite joins the queue automatically. Scan mode is used as an idle-time activity logger: it accumulates packets across many low-priority targets without operator intervention.

The switch between modes is non-disruptive: a track-to-scan change, for instance, completes the current pass before rebuilding the schedule.

5.13 The HTTPS Dashboard

The dashboard is a self-contained HTTPS server built on the Python standard library only, no Flask, no Node toolchain, no build step. The server is approximately 2060 LOC; the front end is a single Pi/dashboard.html of approximately 5650 lines (HTML + inline CSS + vanilla JavaScript). The deliberate absence of a build step makes the dashboard deployable as a single git pull.

HTTPS is mandatory because the browser Geolocation API, used to bootstrap the station's GPS coordinates from a phone, requires a secure context. A 2048-bit RSA self-signed certificate is generated on first run via the system `openssl` binary and stored in `~/.station_ssl/`; the operator accepts the browser warning once.

The single-page application has three tabs:

- Track: pass selection (idle) or live tracking (active pass). The two states of this tab are the subject of Figures 5.3 and 5.4 below.
- Control: manual rotator control (numeric AZ/EL goto, six preset positions including park, zenith, and the four cardinals at 45° elevation), a CC1200 register read/write panel, and a buzzer-pattern test grid.
- Settings: GPS entry, WiFi configuration, station identification (name, callsign, SatNOGS station ID, SatNOGS DB API token), antenna parameters, link-budget overrides, and the first-run setup wizard.

5.13.1 Pre-Tracking View: Pass Selection

Figure 5.3 shows the Track tab in its idle state. The polar plot displays the trajectories of all visible upcoming passes (sourced from the cached `/api/passes` payload), colour-coded by satellite. Three operator-facing elements (numbered in the figure) drive the workflow:

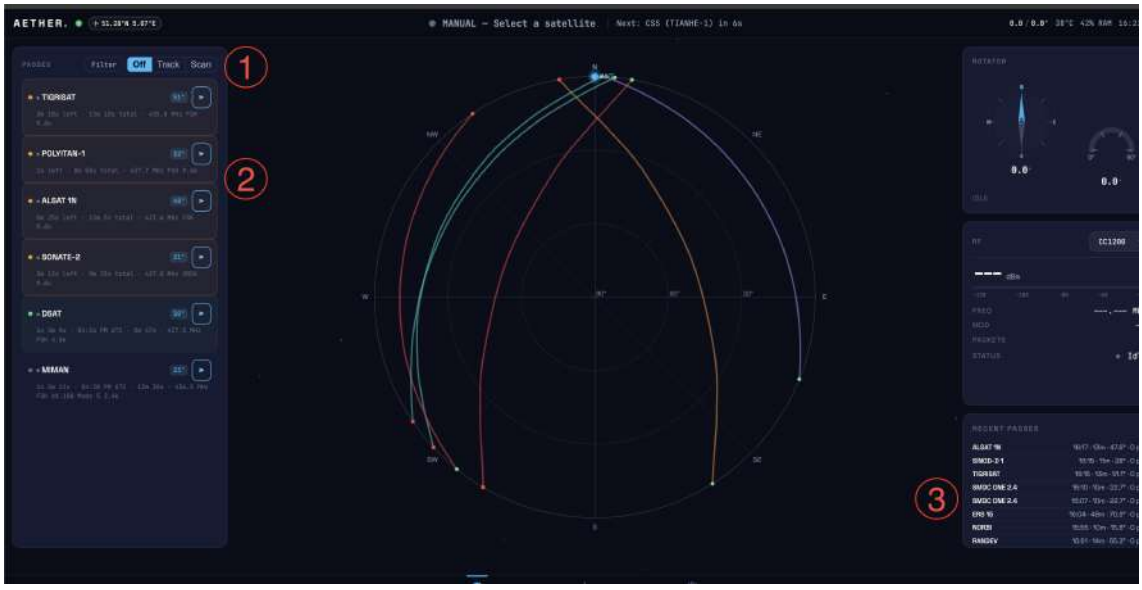

Figure 5.3: Pre-tracking view: filter + mode segmented control (1), upcoming-passes list (2), recent-passes log (3).

Marker 1, filter and mode controls. The `Filter` button (left) opens a panel that constrains

the upcoming-passes list by band (UHF/VHF), receiver type (CC1200 / SDR), minimum elevation, and recent activity. The three-button segmented control (`Off` / `Track` / `Scan`, right) is the daemon's mode selector and is wired to `POST /api/auto-track`, which writes `auto_mode` into `station.conf`; `station.py` re-reads the file every loop iteration. Click semantics:

- Off: manual operation only, the daemon ignores the schedule and parks the rotator. The status banner reads "MANUAL, Select a satellite" (visible in Fig. 5.3).
- Track: auto-track the next pass above the elevation threshold. The banner becomes "AUTO, Waiting" until rise, then "TRACKING, *sat*".
- Scan: enter the multi-satellite scan cycle described in §5.12; the banner becomes "SCAN, *sat*".

Marker 2, upcoming-passes list. Each card is one element of the array returned by `/api/passes`. The dashboard renders the satellite name, the maximum-elevation badge (orange $<30^\circ$, green $>30^\circ$), the time-to-AOS or time-remaining-during-pass countdown, the total duration, and the per-satellite frequency, modulation format, and symbol rate from the resolved `SatProfile`. The play button posts to `/api/track` with the satellite name; `station.py` picks the new target up on its next loop iteration and aborts any current pass for the new target. Countdown timers re-render every 3 s in the browser (`setInterval(applyPasses, 3000)`) so the visible list stays live without re-fetching from the server.

Marker 3, recent-passes log. The last 20 completed passes, served by `/api/pass-history` (refreshed every 30 s). The history file (`~/.station_pass_history.json`) is appended on each LOS by `record_pass_history()` in `Pi/station.py:298`; entries are deduplicated within a 60 s window when the satellite name and `max_el` match (within 1°), so scan-mode visits do not re-record the same pass.

5.13.2 Active-Tracking View: Live Telemetry

Figure 5.4 shows the Track tab during an active AVUM R/B pass. Every numerical and graphical element on the page is driven by the same 2 Hz `/api/status` poll.

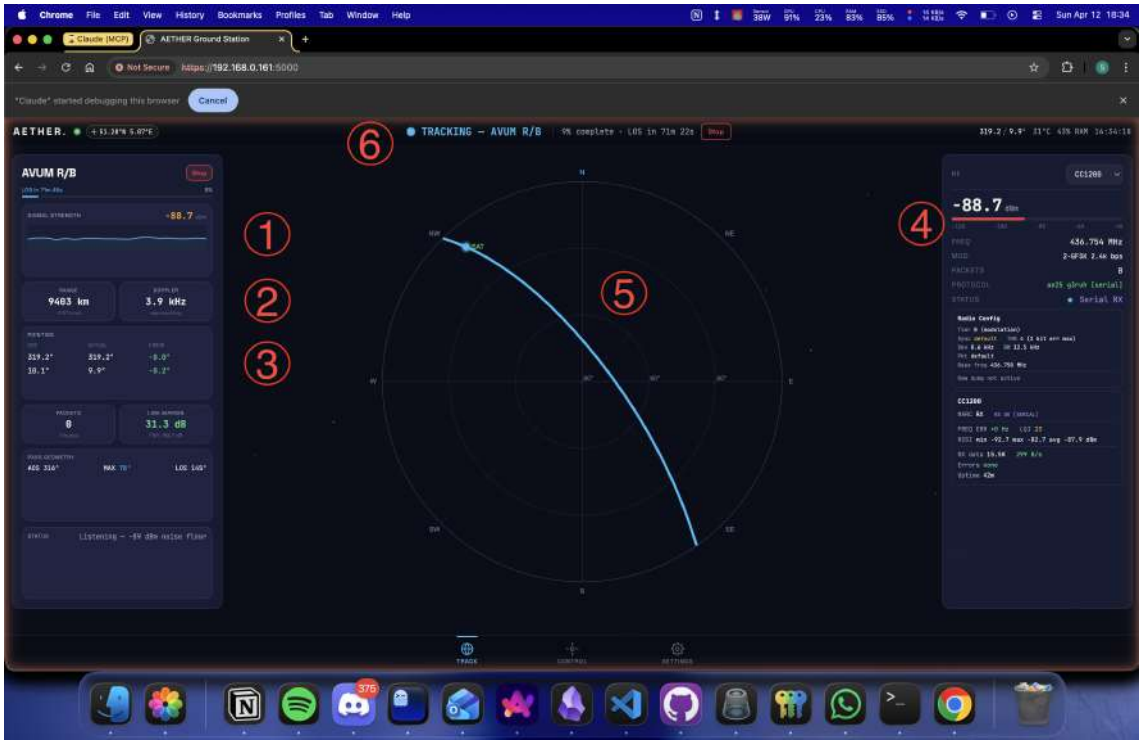

Figure 5.4: Live-tracking dashboard: signal-strength sparkline (1), range/Doppler tile (2), pointing tile (3), RF status panel (4), sky plot (5), top status bar (6).

Marker 1, signal-strength sparkline. The numeric value is `rf.rssi` (in dBm). The CC1200 reports RSSI through the `EXT_RSSI1` (1-dB resolution) and `EXT_RSSI0` (additional 1/8-dB) registers; the RF HAT firmware combines these and reports an integer in 0.1-dB units (`rssi_dbm_x10`) which the Pi divides by 10. The sparkline is a 20-sample rolling history rendered into an HTML5 canvas, with the colour transitioning from green (> -60 dBm) through amber to red (< -80 dBm). Update cadence: 1 Hz from the firmware to `station.py` (`METRICS_INTERVAL_S = 1.0`), 2 Hz onward to the canvas.

Marker 2, range and Doppler tile. `range_km` is `ephem.Body.range / 1000` for the current observer position; the small print under the number is the radial velocity in km/s with a sign convention (negative = approaching, the satellite is closing in). `doppler_hz` is computed by the Doppler equation in §5.10 and reported even when Doppler correction is disabled at the radio side, so the operator always sees the predicted shift. Both values update at the daemon's 10 Hz tick and are sampled by the dashboard at 2 Hz.

Marker 3, pointing tile. A 2×3 grid showing commanded AZ/EL (`cmd_az`/`cmd_el` from `station.py`), actual AZ/EL (from `rotctld`, populated by the dashboard's polling thread), and absolute error per axis. The error column is computed in JavaScript: `|cmd - actual|` for each axis, coloured green ($<0.5°$) / amber ($<2°$) / red. The values in the figure ($-0.0°$ AZ, $-0.2°$ EL during a $9.9°$ EL portion of the pass) are typical of mid-pass tracking and demonstrate that the closed-loop rotator is following the orbital trajectory.

Marker 4, RF status panel (right column). The widest information block, rendering the full state of the CC1200:

- RSSI gauge: the same value as marker 1, rendered on a -120 to -40 dBm scale.
- FREQ: current frequency in MHz including Doppler offset (`rf.freq_mhz`).
- MOD: modulation format and symbol rate from `rf.radio_config.modulation` and `symbol_rate_bps`.
- PACKETS: counter of CRC-passing frames received since AOS.
- PROTOCOL: the per-satellite protocol resolved by §5.9, suffixed `[serial]` or `[pkt]` depending on `rx_mode`. Rendered green for known, amber for assumed (`*_assumed`), red for unknown.
- STATUS: a compact readiness word, “Serial RX”, “Idle”, “Configuring”, “Streaming”, derived from the `RfHatManager` state machine.
- Radio Config sub-card: tier (A/B/C), sync threshold, deviation, channel-filter bandwidth, packet-config flags, and the pre-Doppler base frequency. This is the operator’s verification that the resolved profile reached the radio.
- Raw-dump banner: indicates whether raw byte capture is active; “not active” when the standard packet pipeline is running.
- CC1200 sub-card: low-level CC1200 telemetry from `rf.cc1200`: `MARC` state name (looked up against the 24-state main-state-machine table), suffixed `[SERIAL]` when bit 4 of the firmware status flags indicates synchronous serial mode is active; FREQ ERR (the `FREQOFF_EST` read-back, sign-extended to Hz); LQI; rolling RSSI min/max/avg over the pass; cumulative RX byte count and rate (B/s); error counters (`rx_overflow`, `rx_fifo_err`, `tx_fifo_err`); firmware uptime.

Marker 5, sky plot. The polar canvas is rendered entirely client-side. Three layers compose:

1. *Pre-baked trajectory*: the array of $\{az,\ el,\ t\}$ samples pre-computed by `dashboard.py` for the current pass. Drawn as a faint guide polyline.
2. *Live trajectory*: per-tick `cmd_az`/`cmd_el` samples accumulated in browser memory since AOS, drawn with the satellite-coloured stroke. The trail visually confirms the daemon’s command stream matches the prediction.
3. *Overlays*: the satellite marker (a coloured disc with a label), the antenna heading arrow from rotctld, elevation rings at 30° steps, and cardinal labels (N/E/S/W). The polar projection uses $r = 90 - \mathrm{el}$ and $\theta = \mathrm{az}$.

The canvas is redrawn only when a watched value changes, which keeps the per-frame cost below 1 ms even on the Pi 3A+ when the dashboard is loaded locally.

Marker 6, top status bar. The persistent header collapses the most operator-relevant state into one line:

- Logo + connection dot (green when the daemon's status JSON is fresh, red when stale or absent).
- GPS coordinates pill, derived from `location.lat/lon`.
- State word + target satellite, composed from `state` and `rf.satellite`: "TRACKING, *sat*" (active), "SCAN, *sat*" (scan), "AUTO, Waiting" (auto-track armed), "MANUAL, Select a satellite" (idle).
- Pass progress `pass_progress` = $(1 - \text{los\_seconds}/\text{duration}) \cdot 100$ and the LOS countdown.
- Stop button, writes `~/.station_stop`, which the daemon detects on the next tick and breaks out of the pass loop.
- Live AZ/EL readout from rotctld.
- Pi CPU temperature (`/sys/class/thermal/thermal_zone0/temp`, divided by 1000) and RAM utilisation (`/proc/meminfo`, `MemTotal − MemAvailable`).
- Wall-clock time, formatted client-side.

5.13.3 Setup Wizard

The setup wizard has five steps: (1) GPS fix transferred from a phone browser via the Geolocation API and posted to `/api/location`; (2) the operator points the station north and confirms with `Use Phone Compass`, which calls `/api/calibrate-north` and forwards the EasyComm `ZERO` command to the rotator; (3) attach the antenna (the station is already level from IMU homing); (4) select the receiver type (CC1200 default; RTL-SDR scaffolded but not field-validated); (5) ready. Tracking begins automatically after step 5.

5.14 Background Services and Auto-Start

Five systemd units run the station autonomously after boot:

1. `rotctld.service`, hamlib rotator daemon (model 204), listening on TCP 4533. Restarts automatically if the USB connection to the rotator Pico is interrupted.
2. `dashboard.service`, HTTPS dashboard on port 5000. Starts immediately on boot.

3. `station.service`, the tracking daemon. Waits for `station.conf` (written by the dashboard when the operator provides GPS), then continuously fetches TLEs, computes passes, and tracks them in a loop.
4. `wifi-provision.service`, captive-portal SSID/password entry if WiFi credentials are missing.
5. `rfkill-unblock.service`, generated inline by `setup-pi.sh` to clear the default-blocked WiFi on Debian Trixie at boot.

A setup script (`Pi/services/setup-pi.sh`) configures a fresh Raspberry Pi with all dependencies, UART (`enable_uart=1`, `dtoverlay=disable-bt`), the WiFi rfkill workaround, and service installation. The script is safe to re-run.

5.15 SatNOGS Network Integration

The station is registered as station 4712 ("Aether-Basestation") on `network.satnogs.org`. SiDS (Simple Downlink Sharing Convention) telemetry submission to `https://db.satnogs.org/api/telemetry/` is verified end-to-end (HTTP 201 on each CRC-valid frame; `Pi/satnogs.py:104`). Failed submissions (no internet, server error) are persisted to `~/.satnogs_queue.json` and retried by `flush_queue()` between passes.

Because the CC1200 emits decoded packets rather than raw IQ samples, `satnogs-client` (GNU Radio + SoapySDR) cannot be used as-is: that client drives an SDR through GNU Radio flow graphs and decodes in software, which does not fit a hardware modem. Frames are therefore submitted directly to the SatNOGS DB via the SiDS API. The trade-off is that the station appears "offline" on the SatNOGS network observation map (no job-polling heartbeat), even though it submits decoded telemetry. The reasoning behind this deviation is in §1.1 and §2.5.3.

5.16 Field Deployment Workflow

The deployment sequence, measured at $\sim$4 minutes from unpacking to first tracked pass across the three field sessions:

1. Place on tripod, point north, connect 24 V.
2. Pi boots, EL homes via IMU, services start ($\sim$60 s).
3. Phone joins the Pi WiFi (or the captive-portal provisioner).
4. Dashboard $\rightarrow$ "Use My GPS" $\rightarrow$ station location set.

5. Setup wizard: confirm north, attach antenna, select receiver type.
6. Auto-tracking begins; dashboard shows live status.
7. End of session $\rightarrow$ `PARK` $\rightarrow$ shutdown $\rightarrow$ wait 10 s $\rightarrow$ unplug.

The browser runs on a laptop (primary) or phone; SSH is not used.

Figure 5.5: Close-up of the deployed station.

Chapter 6

Results and Validation

6.1 Slew Speed

Slew speed was measured on a single axis, unloaded, at 24 V supply, by commanding the rotator to traverse 360° at a fixed PWM duty and timing the traverse with the onboard encoder tick counter. Four duty levels were tested. Raw data and per-run encoder-derived rates are listed in Appendix C.

Duty (%)	Speed (°/s)	360° time (s)
35	11.2	32.1
55	18.9	19.0
75	26.9	13.4
95	35.0	10.3

Table 6.1 Slew-speed measurements, unloaded, 24 V supply.

Linear regression over the four measured points gives slope 0.40°/s per % duty, $R^2 > 0.99$. 100 % duty was tested separately and trips the TB6642FG overcurrent protection on motor stall, so it is not included in the fit. The 35°/s ceiling at 95 % duty exceeds the total angular rate $\dot{\theta}_{\max} \approx 1.1$°/s derived in §2.1 by a factor of approximately 32. It does not cover the instantaneous *azimuth* rate during near-zenith passes (which can exceed 10°/s for a few seconds); §6.3 reports the resulting AZ tracking error during such passes.

6.2 PID Step Response

With the deployed gains ($K_p = 0.15$, $K_i = 0.03$, $K_d = 0.02$, duty-filter $\alpha = 0.15$, max duty 0.95; §5.1) a 90° commanded step settles to within the 0.05° deadband in approximately

4 s, with overshoot below 0.5°. The settling time decomposes as: slew-limited traverse of 90° at 35°/s peak rate = $90/35 = 2.6$ s, plus roughly 1.4 s of final convergence through the duty output filter and deadband. The closed-loop bandwidth is bounded above by the duty-output filter time constant:

$$\tau_{\text{duty}} = \frac{\Delta t}{\alpha} = \frac{0.01}{0.15} \approx 0.067 \text{ s}, \qquad f_{-3\,\text{dB}} \approx \frac{1}{2\pi\tau} \approx 2.4 \text{ Hz}. \tag{6.1}$$

In practice the slew-rate limit sets the large-signal response time; the 2.4 Hz figure applies to small-signal tracking above the deadband. Either is well above the $\leq$1.1°/s total angular rate of a 400 km LEO pass (§2.1).

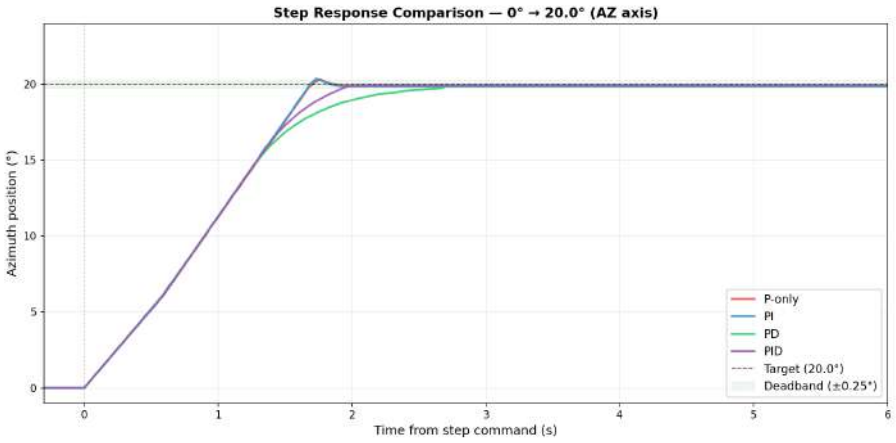

Figure 6.1: PID step response for a 90° commanded step.

6.3 Satellite Tracking Accuracy

Tracking error was computed per pass from the CSV log (commanded AZ/EL from `station.py` pass predictor vs. actual encoder readback from the MotorPCB, sampled at 5 Hz; logs in `captures/20260409/pass_logs/` and `captures/20260412/`). Absolute-value errors are binned by the satellite's peak-elevation category:

Scenario	Error (|commanded − actual|)
EL tracking, all passes	$<$0.5° throughout the pass
AZ tracking, 20°–70° peak elevation	0°–3° throughout the pass
AZ tracking, peak elevation $>$70° (near-zenith)	11°–166° during the $\sim$2–4 s zenith-crossing window; recovers to 0°–3° below 70°

Table 6.2 Tracking error binned by satellite peak elevation.

Across passes with peak elevation in the 20°–70° band (the range where the Chapter 2 link budget closes), both axes remain inside the Siretta Oscar 44 3-dB beamwidths (54° H-plane, 48° E-plane; §1.1). The near-zenith AZ excursions exceed the 35°/s motor ceiling from §6.1 for a few seconds per pass; this is a speed-limit event, not an accuracy-limit one, and the error decreases monotonically as the satellite descends below 70° elevation.

6.4 RF Front-End Measurements

Chapter 2 derived a theoretical link margin of -5.7 dB at 30° elevation and predicted that board-level interference would degrade this further. The measurements in this section and §6.5 quantify the actual outcome.

Two devboards were connected to a host PC and exercised with `RF_Devboard/PC_test/cc1200_rf_test.`
The test script loads a bench validation SmartRF profile (2.4 kbps 2-GFSK, 5.2 kHz deviation, 10 kHz channel filter; modulation index $h = 2\Delta f / f_b \approx 4.3$, chosen for robustness at short range rather than spectral efficiency, this is not the AetherSpace flight configuration, which remains to be defined once the satellite's modulation parameters are finalised), tunes both boards to a specified frequency, and runs a bidirectional packet exchange: Board A transmits a sequence of fixed-size payloads while Board B receives, then the roles are reversed. Payload matching is verified byte-for-byte.

The frequency sweep covered 430–439 MHz in 1 MHz steps with 50 packets per direction per frequency (1000 total). Test conditions: 1 m board separation, dipole antennas, indoor lab, CRC-16 enabled. Table 6.3 shows the per-frequency results averaged over both directions.

Table 6.3 Devboard bench test results per frequency, averaged over both directions (50 packets each). LQI lower = better.

Freq (MHz)	RSSI avg (dBm)	LQI avg	Latency avg (ms)	Loss (%)
430	−83.6†	0.4	103.5	0.0
431	−103.7	0.3	103.3	0.0
432	−102.4	0.5	103.3	0.0
433	−103.6	0.4	104.1	0.0
434	−103.7	0.4	104.0	0.0
435	−88.8†	0.5	106.5	0.0
436	−103.2	0.4	104.8	0.0
437	−103.9	0.5	103.5	0.0
438	−103.9	0.4	104.3	0.0
439	−103.7	0.4	103.6	0.0

† Fractional-N PLL low-dither artefact; see text.

Zero packet loss and 100 % payload match across all 10 frequencies confirm that the CC1200 SPI driver, SmartRF profile loading, COBS framing, TX and RX FIFO paths, sync-word detection, CRC validation, and bidirectional TRX switching all function correctly on the devboard hardware. LQI is a CC1200 demodulator quality metric that accumulates constellation error over 64 symbols after sync detection; lower values indicate smaller error. The uniform averages of 0.3–0.5 across all frequencies reflect near-perfect symbol recovery at short range, and confirm that neither the RSSI anomalies at 430 and 435 MHz nor the noise-floor readings elsewhere affect demodulator performance. Latency averages of 103–107 ms are consistent across the sweep, reflecting the end-to-end pipeline from host TX command through USB CDC, CC1200 transmission, and USB CDC return; the flat profile confirms no frequency-dependent timing effects.

The elevated RSSI at 430 and 435 MHz is a measurement artefact rooted in the CC1200's fractional-N PLL. In a fractional-N synthesiser the divider ratio is not an exact integer, so a sigma-delta modulator continuously switches the divider between neighbouring integer values, a process called dithering, which spreads quantisation noise across a broad frequency range and produces typical phase-noise behaviour. At 430 MHz the FREQ register evaluates to an exact integer multiple (FREQ2 = 0x2B, FREQ1 = FREQ0 = 0x00), placing the synthesiser at a true integer-N boundary where the sigma-delta is idle and dithering does not occur. At 435 MHz the fractional part is exactly one-half (FREQ2 = 0x2B, FREQ1 = 0x80, FREQ0 = 0x00), so the sigma-delta alternates between only two adjacent divider states rather than spanning a broader range; dithering is minimal rather than absent. Both conditions produce lower-than-typical local-oscillator phase noise, which,

combined with the RSSI sample being taken immediately after packet reception before the running average has decayed to the ambient floor, produces the apparent peaks.

What the bench test does not establish is satellite reception. The devboard was not taken outdoors or connected to a directional antenna. Link-budget analysis (§2.3) shows that receiving the AetherSpace downlink requires a front-end LNA regardless of which board is used; the bench test scopes the transceiver stack in isolation, not the system sensitivity in a satellite geometry.

Doppler correction. The Pi computes the Doppler shift at 2 Hz from the TLE-derived radial velocity and sends frequency-word commands to the RF HAT; the firmware applies small deltas via `FREQOFF` and rewrites the full `FREQ+FS_CFG` word on large or forced retunes, with `FREQOFF` also read back as a residual-error estimate. The observed $\pm$9 kHz sweep at 432 MHz is consistent with the $\pm$11 kHz maximum at 433 MHz predicted in §2.4 (the commanded offset is clipped at the mid-pass peak that the satellite actually reaches).

VHF. Not populated on the assembled board; not tested.

6.5 Packet Reception: Result and Interpretation

As of April 2026, no protocol-valid frames have been decoded from orbit. Across three field-test sessions the station tracked approximately 210 satellite passes (16 on April 1 baseline with uncalibrated RSSI readout, 184 on April 9, and 10 on April 12) and captured tens of megabytes of raw binary data, but:

- Zero CRC-16/CCITT-valid HDLC frames (with or without G3RUH descrambling).
- Zero AX.25 frames with valid FCS.
- Zero AX.100 Mode 5 frames passing Reed–Solomon RS(255,223) correction.
- Zero decoded packets submitted to the SatNOGS DB from real satellites.

The receive chain has been validated for local test frames in a short-range ground-to-ground configuration between two devboards (§6.4): transmitted test frames decode at the receiver with valid sync word and zero payload errors across all tested frequencies, exercising the RF path, SmartRF register set, FIFO drain, and host-side decoders under those conditions. This ground validation confirms the digital and RF path is internally consistent; it does not establish that the system is receiving orbital signals.

The April 1 session predates the RSSI-readout fix (AGC_GAIN_ADJUST calibration) and is used as the negative control in the analysis below; the April 9 and April 12 sessions use the post-fix calibration.

6.5.1 Reception Modes Deployed

Two RX modes were used, selected per satellite by `station.py` (§5.7):

1. FIFO mode (default): CC1200 matches a sync word (e.g. AX.100 ASM `0x930B51DE`), then delivers fixed-length 255-byte chunks. If sync or modulation parameters don't match, the FIFO drains receiver noise after false sync triggers.
2. Serial mode with AX.25 G3RUH decode: for $\geq$4800 bps FSK, the firmware switches to PIO serial capture and the G3RUH / NRZ-I / HDLC pipeline of §5.7.1. No CRC-valid AX.25 frames were recovered from orbital data.

6.5.2 Why no decoded frames

The §2.3 link budget shows the AetherSpace downlink is already marginal at the datasheet CC1200 sensitivity and -20 dB at the measured effective sensitivity. CRC-16 is error detection, not correction; RS(255,223) cannot compensate for a deficit of that size. The zero-frame outcome is the predicted outcome.

Three factors contribute:

1. Front-end noise figure and board interference. No LNA, so the CC1200 AGC operates on antenna noise plus on-board interference (LMR51420 buck harmonics, RP2040 digital, Pi GPIO/HDMI); measured noise floor sits approximately 30 dB above the expected kTB+NF limit at the CC1200 input.
2. Modulation parameter uncertainty. The SatNOGS DB does not always publish exact symbol rate / deviation / sync word for non-AetherSpace satellites; any mismatch in FIFO mode produces zero output.
3. Link geometry. A +14 dBm / +2 dBi CubeSat leaves negative theoretical margin at mid-elevation even with a perfect receiver.

6.5.3 What was observed: suggestive but not diagnostic

The captures contain statistical patterns that are individually compatible with weak sub-threshold satellite reception, but each is also compatible with receiver noise or demodulator artefacts triggered by RF activity at the sync-word matcher. Both hypotheses are named explicitly below.

A. Elevation-binned mean RSSI (suggestive). Grouping 62 tracked passes by peak elevation, mean peak RSSI increases monotonically by +4.2 dB from the 0–20° bin to the 60–90° bin. Weak satellite RF entering the front end at higher elevations is one possible

explanation. However, the dataset is also confounded by pointing-dependent antenna temperature, local 433 MHz interference, AGC behaviour, and pass-selection effects; a passive antenna pointed at cold sky would normally lower the noise floor relative to the horizon, so this trend cannot be treated as proof of satellite reception without calibrated controls.

Full numerical analysis: `captures/evidence/signal_evidence.md`.

B. Cross-capture byte matching on SNUGLITE-II (suggestive, inconclusive). Four independent SNUGLITE-II captures contain shared subsequences up to 81 bytes long. Two hypotheses are compatible: (i) repeated beacon frames captured at different orbital points, and (ii) a CC1200 demodulator/idle pattern emitted repeatedly at false sync triggers. The April 9 log explicitly identifies a 99-byte CC1200 idle pattern appearing identically across all satellites at all frequencies; the SNUGLITE-II matches have not yet been shown to differ structurally from this idle pattern, so (ii) is not excluded.

C. Pass-profile RSSI range (weak). A small subset of passes show an AOS-to-peak-to-LOS RSSI excursion of several dB, loosely consistent with $1/r^2$ path-loss modulation of a real signal but equally consistent with antenna noise-temperature variation as the beam sweeps across ground and sky. Not diagnostic. Raw per-pass numbers: `captures/evidence/signal_evidence.md`.

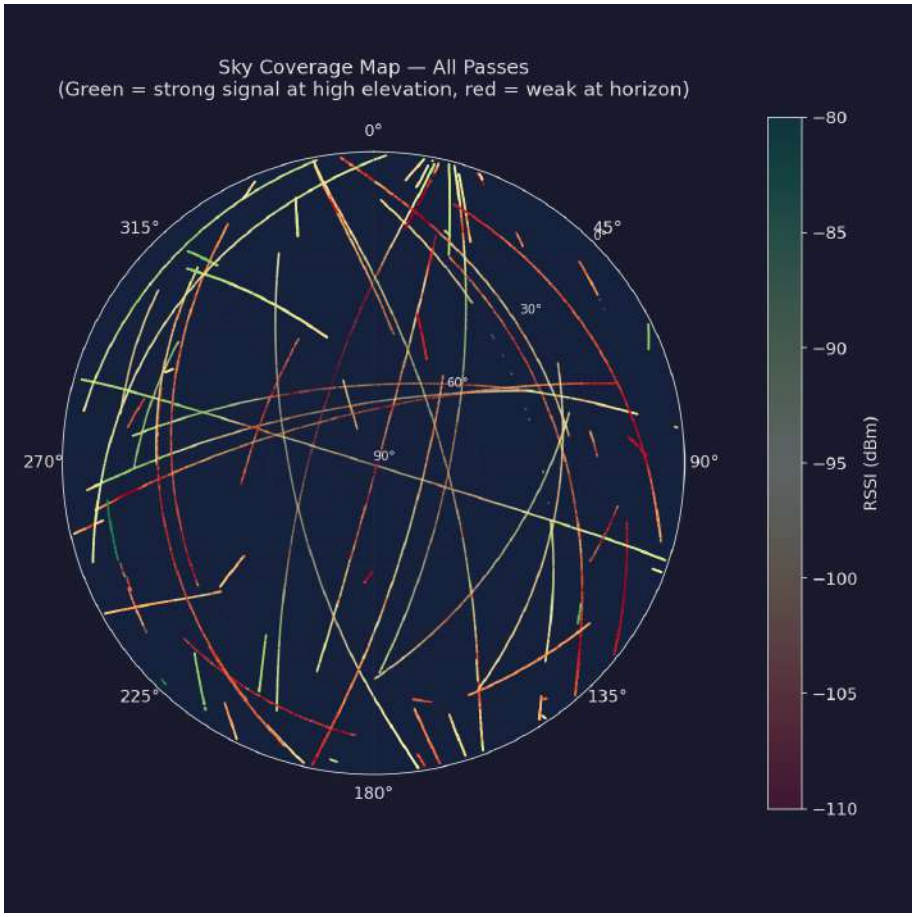

Figure 6.2: Sky coverage heatmap: RSSI over AZ/EL across all tracked passes.

D. Doppler-sweep profile (confirms orbital geometry, not reception). The expected $\pm$9 kHz sweep at 432 MHz with zero-crossing at TCA is observed on every pass. Because this sweep is *applied* by station software to the FREQOFF register regardless of whether any signal is present, it verifies correct predict/correct software behaviour only.

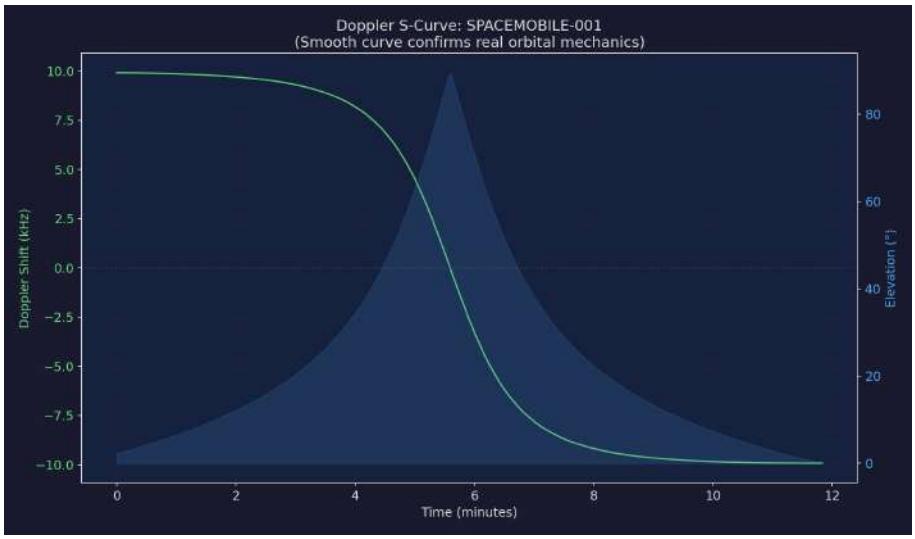

Figure 6.3: Doppler-correction S-curve applied during one pass.

E. Capture-byte entropy (weak). Captures show byte entropy of 6.95–7.91 bits/byte, below the 8.0 bits/byte expected of uniform random noise. Filtered or correlated noise from a demodulator can produce similar signatures without encoding any information. The entropy is consistent with "not white noise" but not with "structured satellite telemetry" specifically.

F. MIMAN AX.100 Mode 5 analysis (inconclusive). MIMAN captures processed through AX.100 Mode 5 achieved Golay(24,12) decode rates of 73 % (April 9) and 66 % (April 12). The random false-positive baseline for Golay(24,12) with 3-error correction is approximately 57 %, so the observed rates are only marginally above chance. Zero frames passed the subsequent RS(255,223) correction. Golay matches without downstream RS-decoded content are not a definitive indicator. See `captures/evidence/miman_decode_report.json`.

6.5.4 Academic Conclusion

The primary finding is established independently of observations A–F: the link budget in §2.3 shows a -20.7 dB measured margin at 30° elevation with the assembled hardware (no LNA), making protocol-valid decode physically impossible regardless of which secondary hypothesis explains the statistical patterns. The secondary finding is that A–F are individually consistent with both weak sub-threshold reception and receiver-internal noise artefacts, and cannot be used to distinguish between the two hypotheses. This ambiguity is expected and does not alter the root-cause diagnosis: the measured board-noise floor sits approximately 30 dB above the thermal + NF baseline, and the required hardware intervention is a front-end LNA (§7.3).

6.5.5 AetherSpace-Specific Outlook

For AetherSpace the modulation-match risk is lower than for arbitrary satellites because the CubeSat and the ground station use the same CC1200 transceiver, provided both sides load the same finalised SmartRF register set. The remaining risk is sensitivity, addressed in §7.3. The software architecture already exposes an `RfBackend` abstraction with both CC1200 and RTL-SDR implementations, allowing an SDR-based wideband IQ capture to serve as an independent verification path once an LNA is installed.

6.6 Field Tests and Link Budget

Three field sessions were conducted with the complete station on a camera tripod in an open field, powered from a bench supply via extension cord and using an iPhone hotspot for internet. The session calendar is given in §6.5; total setup time from unpacking to first tracked pass was measured at approximately 4 minutes, with no laptop or SSH session used.

Figure 6.4: Wide shot of the deployed station in the field.

The canonical link-budget table is in §2.3. The measurement-derived numbers that tie it to this chapter:

- Measured effective noise floor at the CC1200 input on the assembled RF HAT: approximately -96 dBm (canonical figure from `captures/evidence/signal_evidence.md`). This is approximately 30 dB above the expected kTB+NF limit of ~ -127 dBm and is attributable to board-level interference (LMR51420 buck harmonics, RP2040 digital noise, Pi GPIO emissions) and antenna noise temperature. Pure thermal kTB at 290 K for the CC1200 channel bandwidth is ~ -134 dBm; the ~ -127 dBm figure already folds in the CC1200 noise figure.
- Because the system is interference-dominated rather than thermally limited, the primary benefit of a mast-mounted LNA is that its gain ($\sim$20 dB) lifts the received antenna signal above the fixed board-interference floor before it reaches the CC1200, improving the signal-to-noise ratio at the CC1200 input by approximately the LNA gain. A secondary benefit is the reduction of the thermal noise figure from $\sim$7 dB (CC1200 alone) to $<$1 dB (LNA-dominated Friis cascade). The 7 dB figure is derived from the datasheet sensitivity: at 1.2 kbps with an 11 kHz channel filter, $kTB = -174 + 10\log_{10}(11000) = -133.6$ dBm; non-coherent FSK requires $E_b/N_0 \approx 13$ dB, which converts to a bandwidth SNR of $13 - 10\log_{10}(B/R_b) = 13 - 9.6 = 3.4$ dB; so NF $= -123 - (-133.6) - 3.4 \approx 7$ dB. The combined effect improves effective sensitivity by $\sim$20 dB, which would move the AetherSpace 30°/600 km link margin from the measured ~ -21 dB towards break-even.
- A higher-gain Yagi contributes additively but is secondary: the dominant gap is the missing front-end gain, not antenna gain.

CSV pass logs (timestamp, commanded AZ/EL, encoder AZ/EL, RSSI, frequency, Doppler offset, slant range, packet count, 5 Hz cadence) were written to `captures/20260409/pass_logs/` and `captures/20260412/`. Total: $16 + 184 + 10 = 210$ files across the three sessions. Tracking-accuracy and Doppler-sweep results derived from these logs are reported in §6.3 and §6.4 respectively and are not repeated here.

6.6.1 Firmware bugs surfaced by live passes

Four firmware/software defects were diagnosed during field sessions and fixed in the same session:

- AZ integral windup after zenith crossing. When the AZ axis saturated during a high-rate zenith crossing, the PID integral accumulated a large error; after the satellite descended and the rate dropped, the integral fought the proportional term and oscillated. Fixed with the saturation-based anti-windup described in §5.1: the integral stops accumulating when $|u| \geq 0.95$ and the error sign would grow it further.

- Duty output filter too slow. The PWM output LPF at $\alpha = 0.05$ (time constant 0.2 s) made the AZ axis unresponsive at tracking rates below $\sim$1°/s. α was raised to 0.15 (0.067 s) with no observed increase in motor-current transients; this is the deployed setting used throughout §6.2.
- Doppler base-frequency compounding. A defect in `set_frequency()` overwrote the per-satellite base frequency with the Doppler-corrected value, so each successive 2 Hz update accumulated on top of the last correction. Fixed by introducing a separate `uhf_base_freq_hz` variable that holds the satellite's nominal frequency constant across the pass, with the Doppler offset applied only when writing the CC1200 FREQ/FREQOFF registers.
- Rotator socket timeout. The TCP `set_position()` call to `rotctld` blocked up to 5 s on the receive side, limiting effective tracking rate. Reduced to 0.5 s with fire-and-forget sends, plus a 15 s keepalive ping during AOS wait to keep the socket responsive. Implementation in `station.py`.

6.7 Cost Comparison

The v2 MOTOR_RF_HAT BOM (from §3.6) compared against commercial and existing open-source alternatives. The "rotator" column counts the controller board + motors + mechanics only, excluding the station computer, antenna, coax, power supply, and tripod; "total" adds a Pi 3A+.

System	Type	Rotator (€)	+Pi (€)
Yaesu G-5500	Commercial	760–900	1200+
SPID RAS	Commercial	1200–1300	1800+
SatNOGS v3	Stepper, belt	300–500	600+
This work (v2)	**DC + gears**	**$\sim$75**	**$\sim$100**

Table 6.4 Cost comparison. v2 is design-complete but not yet fabricated; $\sim$€75/€100 are BOM projections (LCSC + JLCPCB, April 2026).

Chapter 7

Conclusion and Future Work

7.1 Research Questions Answered

1. Under €50 rotator / €100 station? Partially. v2 (§3.6) reaches $\sim$€40 for the combined controller/RF PCB, $\sim$€75 for the full rotator hardware, and $\sim$€100 for the station core (excluding antenna, coax, power supply, tripod). The strict "full rotator under €50" target is not met once motors and mechanics are included. The v1 setup used for field validation costs $\sim$€145 (two separate boards, 6-layer RF HAT).

2. Pointing accuracy? Measured: $<0.5^\circ$ EL and 0°–3° AZ in the 20°–70° peak-elevation band (§6.3). Encoder resolution 0.00409° AZ, 0.00793° EL (§3.2.3). Both axes stay inside the Siretta Oscar 44 3-dB beamwidths (54° H, 48° E). The large AZ excursions at $>70^\circ$ elevation are a motor-speed-limit event, not a control-loop accuracy limit.

3. SatNOGS integration? Yes on the rotator side via standard EasyComm III / hamlib model 204, and on the telemetry side via SiDS (station 4712 registered). The CC1200 is incompatible with the SDR-based `satnogs-client`, so the station does not appear "online" on the observation-scheduling map; this is a deliberate choice, not a defect.

4. DC vs stepper? DC motors were chosen (rationale in §3.3). Empirical results: zero motor current draw when stationary, $<0.5^\circ$ EL / 0°–3° AZ tracking in the link-closing elevation band, 35°/s peak slew at 95 % duty. Cost: 100 Hz PID with quadrature decode, running within the RP2040 timing budget (ISR latency <10 μs; inter-edge period 117 μs at 35°/s).

5. Battery feasible? Architecturally yes (zero motor current when stationary, 24 V input accepts a 6S LiPo). Analytical projection: $\sim$3.5 W idle + $\sim$18 W peak active, weighted $\sim$90 % idle $\rightarrow$ 20–28 h on a 6S 6 Ah (133 Wh) pack. Not field-validated; see §7.3.

Figure 7.1: 24 V 6S LiPo pack intended for portable operation.

7.2 Limitations

- No protocol-valid frames decoded from orbit: documented in §6.5 with per-observation alternative hypotheses; root cause identified in §2.3 and §6.6 as the absence of an external LNA combined with a board-noise floor approximately 30 dB above the expected kTB+NF limit, degrading effective CC1200 sensitivity from the datasheet -120 dBm to approximately -105 dBm and leaving the AetherSpace link at approximately -20 dB measured margin at 30° elevation (best case; typical outdoor conditions are worse).
- No calibrated-source RF verification against a signal generator. Sensitivity has been validated in relative terms (ground-to-ground CC1200-to-CC1200 decode, §6.5) but not against a signal generator injecting a known power level into the SMA. A sig-gen sweep would separate "receive chain works but insensitive" from "receive chain has an undiagnosed defect" independently of link geometry.
- VHF path not validated. The 144/145 MHz front-end is routed on the RF HAT but not populated. It would need a VNA-based matching-network tune before uplink use.
- Zenith AZ rate saturation. The motor's 35°/s ceiling is exceeded briefly during high-elevation passes (near-zenith passes can demand $>10^\circ$/s of *azimuth* rate for a few seconds, even though the total angular rate never exceeds $\sim 1.1^\circ$/s). The AZ axis saturates during this window and recovers as the satellite descends below $\sim 70^\circ$ elevation.
- First-rev PCB issues: bodge wires (AZ IN1 rerouted GP15$\rightarrow$GP16), replaced TB6642FG, removed encoder RC caps. All addressed in the v2 MOTOR_RF_HAT design.

- Design-complete v2 not yet built. The cost claim relies on v2 BOM pricing; v2 has not been fabricated in time for this thesis. The v2 design incorporates every lesson from v1 bring-up and is ready to order.

7.3 Future Work

1. Build the v2 MOTOR_RF_HAT: fabricate and assemble the design-complete combined board. Validates the $\sim$€100 station-core cost projection end-to-end and removes the v1 bodges.
2. External mast-mounted LNA: the single most impactful reception change. Concrete recommendation:
   - LNA: Qorvo QPL9547 (NF $\sim$0.3 dB, $\sim$20 dB gain at 433 MHz, +5 V) as a modern replacement for the discontinued SPF5189Z. Mini-Circuits PGA-103+ (NF $\sim$0.6 dB, $\sim$20 dB gain at 433 MHz) is an equivalent in-stock part. There is no TI companion LNA for this band; the CC1190 range extender covers 850–950 MHz only.
   - SAW pre-filter (optional but recommended): 433.92 MHz SAW bandpass before the LNA, to reject out-of-band ISM / PMR / ATV interferers that otherwise compress the LNA and lift the AGC.
   - Architecture: mast-mount the LNA, fed over the antenna SMA coax via a bias-T on the RF HAT (a 470 nH–1 μH RF choke plus DC-block capacitor; the choke reactance must be well above $Z_0 = 50\ \Omega$ at 433 MHz to keep insertion loss negligible: 470 nH gives $X_L \approx 1.3$ kΩ, yielding $\approx$0.2 dB loss; 120 nH gives only 326 Ω and $\approx$0.6 dB, which is significant ahead of a low-noise amplifier). Placing the LNA at the antenna feed rather than after the coax makes the LNA's NF set the whole-chain NF (Friis cascade) and pre-amplifies before the 0.5–1 dB RG-58 loss and the 15–20 dB board-noise elevation measured on v1.
   - Antenna (optional secondary upgrade): a 10–12 dBi Yagi in place of the Oscar 44 adds a few dB, but the dominant gap is the LNA and the antenna upgrade is not required to close the link.

   Combined impact (LNA alone, existing Yagi): system NF $\sim$7 dB $\rightarrow$ $<$1 dB, AGC lifts off the board-noise floor. Effective sensitivity improves by 15–20 dB, moving the AetherSpace link at 30° elevation from -20.7 dB measured to roughly $0 \ldots +5$ dB. v2 PCB change: add a bias-T network on the UHF SMA port; the LNA itself stays off-board.
3. AetherSpace integration: once AetherSpace's final packet format is published, load the matching SmartRF profile. A CC1200-to-CC1200 link removes a large part of

the modulation-parameter uncertainty if both ends use the same register profile; the open question is sensitivity, addressed by (2).

4. RTL-SDR hybrid: add an RTL-SDR for wideband IQ capture and visual waterfall. Useful as an independent verification path for CC1200 reception claims. Software architecture already supports it via the abstract `RfBackend` interface.
5. VHF uplink: populate the 144/145 MHz matching network, VNA-tune, and enable uplink commanding once AetherSpace defines an uplink protocol.
6. Battery field test: validate the 20–28 h runtime projection with current measurements at each operating point on a representative 6S pack.

7.4 Summary

This thesis designs, builds, and partially validates a portable SatNOGS-compatible ground station. The v1 hardware (two separate boards, used in the field sessions) tracks satellites with sub-0.5° elevation error and 0°–3° azimuth error in the 20°–70° peak-elevation band, integrates with the SatNOGS rotator ecosystem through standard EasyComm III, and deploys from unpacked to tracking in about four minutes using only a laptop (a phone can also supply GPS in a pinch). The v2 MOTOR_RF_HAT (design-complete, unfabricated at submission) projects to ∼€40 for the combined controller/RF PCB and ∼€100 for the station core; the projection remains to be validated by fabrication.

The main open gap is reception. No protocol-valid frames were decoded from orbit across approximately 210 tracked passes; the root cause is documented in §6.5 and §2.3 as the absence of an external LNA combined with a measured board-noise floor approximately 30 dB above the thermal+NF limit, and the mitigation is specified in §7.3 item 2.

Relative to Hansen[illegible]s (2023) [12], this thesis contributes: (a) a built and field-validated 3D-printed AZ/EL rotator, (b) a custom RP2040 motor-controller firmware with closed-loop PID, IMU-based homing, and runaway safety, (c) a dual-CC1200 Raspberry Pi HAT with a design-complete combined v2 variant consolidating motor control and RF front-end onto one 6-layer board, (d) a ∼7 900 LOC Python stack covering pass prediction, Doppler correction, per-pass CC1200 reconfiguration, and SatNOGS DB telemetry submission, (e) a zero-SSH laptop-based field-deployment workflow, and (f) a first-principles and measurement-grounded link-budget analysis that quantifies the sensitivity gap blocking orbital decode and names the specific front-end change (LNA) that would close it.

Bibliography

[1] O. Montenbruck and E. Gill, *Satellite Orbits: Models, Methods, and Applications*. Berlin: Springer, 2000.

[2] Siretta, *OSCAR-44 5-element Yagi antenna, 400–480 MHz datasheet rev 1.0*, Siretta Ltd., 2024.

[3] Yaesu. “G-5500 azimuth-elevation rotator system”. Product datasheet. [Online]. Available: https://www.yaesu.com/

[4] RFHAM. “SPID antenna rotator systems”. [Online]. Available: https://www.rfhamdesign.com/

[5] SatNOGS Wiki. “SatNOGS rotator v3”. [Online]. Available: https://wiki.satnogs.org/SatNOGS_Rotator_v3

[6] SatNOGS Wiki. “SatNOGS rotator v3 — bill of materials”. [Online]. Available: https://wiki.satnogs.org/SatNOGS_Rotator_v3/Bill_of_Materials

[7] Texas Instruments, *CC1200 low-power, high-performance RF transceiver*, Datasheet SWRS123D, 2014.

[8] Libre Space Foundation. “SatNOGS — open source ground station network”. [Online]. Available: https://satnogs.org/

[9] Hamlib Project. “Ham radio control libraries”. [Online]. Available: https://hamlib.github.io/

[10] C. (Jackson, *EasyComm — a simple satellite rotator protocol*, Original EasyComm I/II specification, 1990. [Online]. Available: https://www.mustbeart.com/software/easycomm.txt

[11] Raspberry Pi Foundation. “Raspberry Pi 3 Model A+ datasheet”. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-3-model-a-plus/

[12] S. Hanssens, “Communicatie van een RF-grondstation met de AetherSpace CubeSat: Analyse, ontwikkeling en implementatie van een SatNOGS grondstation”, Draft, M.S. thesis, KU Leuven, Campus Geel, 2023.

[13] ITU-R, *Recommendation ITU-R P.676 — attenuation by atmospheric gases and related effects*, International Telecommunication Union, 2022.

[14] SatNOGS Network. “Station list”. [Online]. Available: https://network.satnogs.org/stations/

[15] SatNOGS Wiki. “SatNOGS main page”. [Online]. Available: https://wiki.satnogs.org/Main_Page

[16] Libre Space Foundation. “SatNOGS rotator firmware — protocol documentation”. [Online]. Available: https://librespacefoundation.gitlab.io/satnogs/satnogs-rotator-firmware/

[17] Libre Space Foundation. “SatNOGS client — system requirements and setup”. [Online]. Available: https://wiki.satnogs.org/Raspberry_Pi_3_Setup

[18] SuperAntennaz, *Heavy-duty DIY rotator design*, Amateur radio community documentation.

[19] University of Patras. “UPSat — open source CubeSat”. [Online]. Available: https://upsat.gr/

[20] TU Delft. “Delfi space program”. [Online]. Available: https://www.tudelft.nl/lr/delfi-space

[21] G. Masera et al., “AraMiS — architecture for modular satellites”, in *IEEE Aerospace Conference*, Politecnico di Torino, 2005.

[22] International Organization for Standardization, *ISO 53:1998 — cylindrical gears for general and heavy engineering – standard basic rack tooth profile*, ISO, 1998.

[23] *FAPG36-555 series motor datasheet*, AliExpress vendor documentation. Motor internal: RS-555 DC motor, 516:1 planetary gearbox, FT-555 Hall encoder.

[24] Pololu. “Stepper motor power consumption notes”. [Online]. Available: https://www.pololu.com/

[25] Toshiba, *TB6642FG brushed DC motor driver datasheet*, 2014.

[26] Analog Devices, *ADXL345 3-axis digital accelerometer datasheet*, Rev. G, 2015.

[27] Texas Instruments, *CC1200EM 420–470 MHz reference design*, Document SWRR122, 2013.

[28] Raspberry Pi Ltd, *RP2040 datasheet*, 2023. [Online]. Available: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[29] Texas Instruments, *LMR51420 2-A, 42-V step-down voltage regulator*, Datasheet SNVSAI3, 2021.

[30] Richtek Technology, *RT6150 0.6 A, 1.3 MHz fixed-frequency buck-boost converter*, 2020.

[31] JLCPCB. “JLC06121H-3313 6-layer PCB stackup specification”. [Online]. Available: https://jlcpcb.com/impedance

[32] D. M. Pozar, *Microwave Engineering*, 4th. Wiley, 2011, ISBN: 978-0-470-63155-3.

[33] JLCPCB. “JLCPCB PCB impedance calculator”. [Online]. Available: https://jlcpcb.com/pcb-impedance-calculator

[34] M17 Project, SP5WWP, and DB9MAT. “CC1200_HAT rev B — open hardware CC1200 RF module”. [Online]. Available: https://github.com/M17-Project/CC1200_HAT-hw

[35] Texas Instruments, *CC1120EM 169 MHz reference design schematic*, Document TIDR220, 2012.

[36] J. Miller, *G3RUH scrambler — AMSAT 9600 baud FSK packet radio*, AMSAT-UK, 17-bit LFSR, polynomial $x^{17} + x^{12} + 1$, used for NRZ-I bit-stuffing on amateur-satellite 9k6 AX.25, 1988.

Appendix A

GPIO Pin Mapping

Motor Controller Pico

GPIO	Function	Connected To	Notes
GP2	NeoPixel data	WS2812B	Relocated from GP16 on rev1
GP4	SPI0 MISO	ADXL345 IMU	
GP5	SPI0 CSn	ADXL345 IMU	
GP6	SPI0 SCK	ADXL345 IMU	
GP7	SPI0 MOSI	ADXL345 IMU	
GP8	AZ endstop	AZ_END	`kAzEndstop`, active-low
GP9	EL endstop	EL_END	`kElEndstop`, active-low
GP10	Encoder AZ-B	Motor AZ encoder	Firmware label `kAzEncB` (A/B swapped for correct count direction)
GP11	Encoder AZ-A	Motor AZ encoder	Firmware label `kAzEncA`
GP12	Encoder EL-A	Motor EL encoder	
GP13	Encoder EL-B	Motor EL encoder	
GP14	AZ motor IN2	TB6642FG U1	
GP16	AZ motor IN1	TB6642FG U1	Bodge wire from GP15 (damaged on rev1)
GP19	AZ motor ALERT	TB6642FG U1	`kAzAlert`, active-low fault input
GP20	EL motor ALERT	TB6642FG U2	`kElAlert`, active-low fault input
GP21	EL motor PWM	TB6642FG U2	
GP22	AZ motor PWM	TB6642FG U1	
GP25	Pico on-board LED	Pico module	`kLed`, active-high
GP27	EL motor IN2	TB6642FG U2	

Firmware uses GP16 for AZ_IN1 after the GP15-pad damage described in §3.5; the schematic still carries the original GP15 label. Encoder A/B pin labels are swapped at the firmware level to invert the count direction on AZ. These schematic-vs-firmware discrepancies are intentional and documented in `Firmware/rp2040-satnogs-rotator/src/pins.h`.

RF HAT Pico

GPIO	Function	Connected To
GP0	UART0 TX	Pi GPIO15 (RXD)
GP1	UART0 RX	Pi GPIO14 (TXD)
GP4	NeoPixel data	4× WS2812B
GP5	Buzzer PWM	Passive piezo (via NPN/MOSFET gate)
GP6	UHF GPIO3	CC1200 UHF
GP7	UHF GPIO0	CC1200 UHF
GP8	UHF RESET	CC1200 UHF
GP9	UHF GPIO2	CC1200 UHF
GP10	SPI1 SCLK	CC1200 UHF
GP11	SPI1 MOSI	CC1200 UHF
GP12	SPI1 MISO	CC1200 UHF
GP13	SPI1 CSn	CC1200 UHF
GP16	SPI0 MISO	CC1200 VHF
GP17	SPI0 CSn	CC1200 VHF
GP18	SPI0 SCLK	CC1200 VHF
GP19	SPI0 MOSI	CC1200 VHF
GP20	VHF RESET	CC1200 VHF
GP21	VHF GPIO2	CC1200 VHF
GP22	VHF GPIO0	CC1200 VHF
GP26	VHF GPIO3	CC1200 VHF

Table A.2 RF HAT Pico pin assignments.

Appendix B

Communication Protocols

EasyComm III (Rotator: USB CDC serial, 115200 baud)

Command	Response	Description
AZxxx.x ELyyy.y	(none)	Set target position
AZ	AZxxx.x ELyyy.y	Query current position
VE	VESatNOGS-RP2040-v0.2	Firmware version (VE prefix inline)
GS	GS<status_reg>	Status register bitmask
GE	GE<error_reg>	Error register bitmask
PARK	AZxxx.x ELyyy.y	Return to (0°, 0°); replies with position
STOP	AZxxx.x ELyyy.y	Halt motors; replies with position
RESET	AZxxx.x ELyyy.y	Start the homing/reset routine; replies with position
RECAL	(none)	Re-read IMU, re-home EL
ZERO	confirmation string	Set current position as zero reference
MONITOR	telemetry stream	Toggle 10 Hz debug (ticks + endstop + alert pins)
TICKS	AZ_TICKS=n EL_TICKS=n	Report raw encoder ticks
KP<f> / KI<f> / KD<f>	(none)	Set individual PID gains at runtime
GAINS	KP=x KI=x KD=x	Read current PID gains
IMU	X=f Y=f Z=f g EL=f deg	Read ADXL345 accelerometer

Table B.1 EasyComm III commands (SatNOGS variant, hamlib model 204).

COBS Binary Protocol (RF HAT: UART 115200 baud)

Frame structure (little-endian): `[type, seq, len_lo, len_hi, payload..., crc16_lo, crc16_hi]`, COBS-encoded with a 0x00 delimiter. CRC: CRC-16/CCITT-FALSE (polynomial 0x1021, init 0xFFFF). Responses use `type | 0x80`; e.g. `PING` (0x01) response is 0x81.

Type	Name	Description
0x01	PING	Connectivity test
0x02	GET_INFO	CC1200 part/version
0x03	SET_STATE	Strobe CC1200 state machine
0x04	SET_STREAMING	Enable/disable continuous RX forwarding
0x05	GET_METRICS	RSSI, MARC state, FIFO levels
0x10	READ_REG	Normal register read
0x11	WRITE_REG	Normal register write
0x12	READ_EXT	Extended register read
0x13	WRITE_EXT	Extended register write
0x20	RX_READ	Read bytes from RX FIFO
0x21	STROBE	Issue raw CC1200 strobe command
0x22	TX_WRITE	Write to TX FIFO
0x23	TX_SEND	Strobe TX
0x24	TX_FLUSH	Flush TX FIFO
0x30	PROFILE_CLEAR	Clear loaded profile buffer
0x31	PROFILE_BEGIN	Start SmartRF profile load
0x32	PROFILE_CHUNK	Profile data chunk
0x33	PROFILE_APPLY	Apply loaded profile
0x34	SET_FREQ_WORD	Atomic Doppler frequency update
0x35	SET_SERIAL_MODE	Raw serial / AX.25 G3RUH PIO mode
0x40	SELECT_RADIO	Switch UHF/VHF
0x50	BUZZER	Play pattern (0x01–0x08)
0xE0	EVT_RX_DATA	Received packet (async, firmware→Pi)
0xE1	EVT_ERROR	Error event (async, firmware→Pi)

Table B.2 COBS binary protocol message types (RF HAT ↔ Pi).

Appendix C

PID Tuning and Speed Test Data

PID tuning parameters

Parameter	Value	Notes
K_p	0.15	Proportional gain
K_i	0.03	Integral gain
K_d	0.02	Derivative gain
Deadband	0.05°	No correction below this error magnitude
Loop rate	100 Hz	Main `for(;;)` loop on core 0, period-gated with `absolute_time_diff_us()`
I-clamp	$\pm$5°·s	Maximum integral accumulation
D-filter α	0.15	First-order LPF on derivative term
Duty filter α	0.15	First-order LPF on PWM output (deployed value; initially 0.05, raised during field testing)
Max duty	95 %	Prevents TB6642FG overcurrent on stall
PWM frequency	20 kHz	Above audible
AZ ticks/deg	244.6	$64 \times 516 \times (40/15)/360$
EL ticks/deg	126.1	$64 \times 516 \times (55/40)/360$
AZ invert	false	
EL invert	true	Motor direction reversed in firmware
Runaway threshold	50°	E-stop beyond soft limit

Table C.1 PID and control-loop constants from `Firmware/rp2040-satnogs-rotator/src/config.h`.

Speed test data

Duty (%)	Speed (°/s)	360° time (s)	RPM at output	Current (A)
35	11.2	32.1	1.9	∼0.3
55	18.9	19.0	3.2	∼0.5
75	26.9	13.4	4.5	∼0.7
95	35.0	10.3	5.8	∼1.0

Table C.2 Slew-speed measurements (single axis, unloaded, 24 V supply), as reported in §6.1.

Appendix D

Detailed Bill of Materials

The BOM below costs the **v2 combined MOTOR_RF_HAT** (design-complete, routed, ready to order, not yet fabricated at the time of submission). Field validation used the v1 two-board setup (separate MotorPCB + RF HAT), which was more expensive because each board was ordered separately and the RF HAT uses a 6-layer stackup. The lessons from v1 bring-up (§3.5) are incorporated into the v2 BOM: A4950 replaces TB6642FG for availability, and four 100 Ω series resistors (R11–R14) protect the A4950 logic inputs from GPIO transients. The off-board rotator BOM (motors, gears, bearings, chassis, fasteners) is unchanged from v1.

Component prices from the LCSC BOM tool (April 2026), hand-assembled. Full machine-readable BOM in `ThesisPaper/bom_complete.csv`.

D.1 MOTOR_RF_HAT PCB: on-board components

125 placed components (excluding test pads and mounting holes), 48 unique LCSC line items.

Category	Key components	Qty	LCSC total (€)
Capacitors	19 values (0603/0805/1206), CC1200 matching + bypass	71	7.41
Inductors	11 values (0603 RF + 3030 power), CC1200 matching	17	1.18
Resistors	7 values (0603)	14	0.07
CC1200RHBR	UHF + VHF transceivers (C122881)	2	4.43
A4950ELJTR-T	Motor drivers AZ + EL (C82404)	2	1.10
LMR51420YFDDCR	24 V $\rightarrow$ 5 V buck (C7296200)	1	0.84
WS2812B-2020	Status LEDs (C965555)	4	0.28
40 MHz crystal	CC1200 reference (C5380316)	2	0.12
BSS138	Level shifter (C112239)	1	0.01
1N4148WS	Protection diode (C2128)	1	0.01
D_Schottky	Buck diode (C3759853)	1	0.14
XT30PW-M	24 V power input (C431092)	1	0.24
2$\times$20 Pi header	GPIO (C50980)	1	0.14
JST XH 6-pin	Motor/encoder connectors (C157919)	2	0.33
Visible line-item sum			16.30
MOQ rounding uplift	(reels of 100 for 1–8-piece needs)		12.14
LCSC subtotal			**28.44**

Table D.1 MOTOR_RF_HAT on-board component cost breakdown.

Items sourced outside the default LCSC reflow batch (hand-soldered or not stocked at LCSC):

Component	LCSC #	Qty	€
SMA edge-mount connector	(not on LCSC)	2	1.50
15 nH 0603 inductor	C23651	2	0.50
Raspberry Pi Pico (hand-soldered)	C7203002	1	4.00
Subtotal			**6.00**

Table D.2 Hand-soldered / off-batch items.

D.2 PCB fabrication

Item	€
Bare PCB (6-layer, JLCPCB, per board from 5-piece order)	6.00

Table D.3 PCB fabrication cost (per board).

D.3 Off-board components

Component	Source	Qty	€
FAPG36-555-EN motor (24 V, 516:1)	AliExpress	2	16.00 (line)
Raspberry Pi 3 Model A+	RPi distributor	1	25.00
ASA filament ($\sim$400 g)	local	1	10.00
Steel balls 3 mm (50 pcs)	AliExpress	1	2.00
M4$\times$10 + M3$\times$8 screws, heat-set inserts	local	lot	3.00
PVC pipe + misc hardware	local	lot	2.00
ADXL345 IMU breakout	AliExpress	1	1.50
Off-board subtotal			**59.50**

Table D.4 Off-board mechanical and computing components.

D.4 Grand total

Subsystem	€
MOTOR_RF_HAT components (LCSC)	28.44
Hand-soldered parts (SMA, 15 nH, Pico)	6.00
Bare PCB (6-layer)	6.00
Off-board components	59.50
Grand total per station core	**$\sim$€100**

Table D.5 v2 grand-total cost (station core, excluding antenna, coax, power supply, and tripod).

Excludes antenna, coax cable, 24 V power supply, and tripod (user-supplied). Hand-assembled, so no JLCPCB PCBA fees. LCSC total includes MOQ rounding (buying 100 passives when 1–8 are needed). The combined MOTOR_RF_HAT board eliminates the separate motor controller PCB and USB cable between Pi and rotator used in v1 testing. Numbers are projections for the unfabricated v2 board.

Appendix E

GenAI code of conduct for students

Generative AI (GenAI) assistance tools can be used to generate text, image, code, video, music, or combinations of these. It includes typical tools like (but this list is not limited to): ChatGPT, Google Gemini, MS Copilot, Midjourney, Claude.ai, Perplexity.ai, Dall-E, etc.

Student Information

Student A: Stan Coene
Student B: forenameB surnameB

This form is related to our joint Master thesis. Both students declare jointly.
Title Master thesis: Design and Implementation of an Economical SatNOGS-Compatible Ground Station for the AetherSpace CubeSat
Promoter: Prof. dr. ing. J. Vanhamel
Co-promoter: Prof. dr. ir. V. De Smedt
Daily supervisor: _____

Use of GenAI Assistance

Please indicate with ”X” (possibly multiple times) in which way you were using GenAI:

- ☐ We did not use any GenAI assistance tool.
- ☐ We did use GenAI assistance. In this case, specify which ones (e.g., ChatGPT, . . .): _____

GenAI Assistance Used As/For

☐ MS Copilot

☐ Language assistance

☐ Search engine

☐ Literature search

☐ Short-form input assistance

☐ Generating programming code

☐ Generating new research ideas

☐ Generating blocks of text

☐ Other (specify): _____

Code of Conduct for Different Uses

Language Assistant for Reviewing or Improving Texts

This use is similar to using spelling and grammar check tools, and you do not have to refer to the use of GenAI in the text. Be careful:

- Using GenAI tools on texts you did not write yourself to cover up plagiarism (by paraphrasing original texts) is not allowed.

Search Engine for Information or Existing Research

This use is similar to a Google search or checking Wikipedia. If you then write your own text based on this information, you do not have to refer to the use of GenAI in the text. Be careful:

- Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information.
- The output may not be entirely correct and is limited due to the databases it uses. Knowledge evolves and may change over time, and it may be that the database of the GenAI tool is not up to date.

Literature Search

This use is comparable to a Google Scholar search. Be careful:

- Be aware that the output is restricted to the database it is built on. After this initial search, look for scientific sources and conduct your own analysis.
- GenAI tools (like ChatGPT) may output no or wrong references. As a student, you are responsible for further checking and verifying the accuracy of references.

Short-form Input Assistance

This use is similar to Google Docs powered by generative language models.

Generating Programming Code

The use of GenAI for coding should be explicitly allowed by the teacher. If used for coding, correctly mention the use of GenAI assistance and cite it.

Generating New Research Ideas

Further verify whether the idea is novel or not. It is likely that it is related to existing work, which should be referenced.

Generating Blocks of Text

Inserting blocks of text without quotes and a reference to GenAI assistance in your report or thesis is not allowed. Be careful:

- When you literally copy elements from a conversation with a GenAI tool, quote them between quotation marks and refer to them according to the specified reference style.
- Describe the use of the GenAI tool (tool name, version, date, etc.) in the method section and optionally add the full conversation as an attachment.

Other

Contact the professor of the course or the promoter of the thesis. Inform also the program director. Motivate how you comply with Article 84 of the exam regulations. Explain the use and added value of ChatGPT or another AI tool.

Further Important Guidelines and Remarks

- GenAI assistance cannot be used related to data or subjects under Non-Disclosure Agreement.
- GenAI assistance cannot be used related to sensitive or personal data due to privacy issues.
- Take a scientific and critical attitude when interacting with GenAI assistance and interpreting its output.
- As a student, you are responsible for complying with Article 84 of the exam regulations: your report or thesis should reflect your own knowledge, understanding, and skills.

Exam Regulations Article 84

“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e., copying the work (ideas, texts, structures, designs, images, plans, codes, . . .) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources.”

Additional Reading

More information about being transparent on the use of GenAI assistance and about citing and referencing GenAI can be found on the student website.

A Few Final Words

If you are uncertain whether or not you should declare your use of AI tools, discuss the matter with your teacher or promoter. It is safer to declare AI use when it is not needed than to withhold that declaration when it is required.

Finally, remember that advanced AI tools are new and that they can do things they could not do until recently. It is important to follow up on the most recent developments in AI technologies and communicate openly with your teachers, assistants, supervisors, and peers.